AMITY UNIVERSITY ONLINE, LUCKNOW UTTAR PRADESH

In partial fulfilment of the requirement for the award of degree of **Bachelor of Computer Application (Data Analytics & Reporting)**

TITLE**: Advanced Analytics for Credit Card Operations**

Guide Details:

Name: Vaibhav Shah

Designation: AVP

Submitted By:

Name of the Student - Kajal Rawal

Enrolment. No: A9922523000249

Advanced Analytics for Credit Card Operations

ABSTRACT

This project is about understanding credit card data and turning it into useful insights using SQL, Python, and Power BI. Today, banks and financial institutions receive a large amount of data from credit card usage. Every swipe, payment, purchase, or interest charge generates information. If this information is analyzed properly, it can help banks understand customers better, improve their services, and increase revenue. That is the main idea behind this project.

The first step of the project was to organize all the data in a proper structure. For this, two SQL tables were created: credit_card and customer. The credit_card table contains all financial and transaction-related details such as annual fees, credit limit, total transactions, interest earned, chip usage, and whether the customer is delinquent. The customer table stores personal and demographic information such as age, gender, education level, income, job type, and satisfaction score. Storing all the data in a well-designed SQL database makes it easier to manage, clean, and analyze.

Once the tables were created, the next step was to clean the data. Real-world data is rarely perfect. Some values may be missing, some records may be incorrect, and formats may not match. SQL queries were used to remove duplicates, fix date formats, correct text fields, and create new calculated columns. Cleaning the data is very important because any error in the data will result in wrong insights later.

Python was used for additional data preparation tasks. It helped identify missing values, fix inconsistent records, and generate basic statistical summaries. Python is very helpful because it can handle large datasets easily and perform deeper checks that may be difficult in SQL. It also

helped in exploring the data before building the dashboard. For example, Python helped understand how many customers belong to different age groups, how income is distributed, and which features show strong patterns. This step made the data more reliable and ready for the dashboard.

After the data was fully cleaned and prepared, Power BI was used to create an interactive dashboard. Power BI allows users to see the information in a visual form instead of reading long tables. The dashboard includes bar charts, pie charts, line charts, KPIs, slicers, and filters. Users can explore trends by card category, age group, gender, education level, and spending type. The dashboard allows users to understand what is happening in the data by simply clicking on a few visuals.

From the dashboard, several meaningful insights can be observed. For example, transaction amount and transaction count help identify heavy spenders. Credit utilization ratio shows how much of the credit limit customers actually use. Interest earned and annual fees show how much revenue the bank is generating. Delinquent accounts help identify risky customers who may not pay their dues on time. Chip usage helps the bank understand whether customers prefer secure transactions. Expense types reveal where customers spend most such as shopping, travel, fuel, food, entertainment, or bills.

The combination of customer data and transaction data gives a clear picture of customer behavior. For example, age and income can be compared with spending habits. Education level and job type can be compared with credit limit and transaction frequency. Satisfaction score helps the bank understand whether customers are happy or need better services. With all these

insights, the bank can create better marketing strategies, design personalized offers, adjust credit limits, and reduce financial risks.

One of the important strengths of this project is that it connects three major technologies: SQL, Python, and Power BI. SQL handles the backend operations, Python manages the data cleaning and exploration, while Power BI creates a user-friendly dashboard. This combination makes the system flexible, accurate, and easy to understand. It also shows how different tools can work together to create a complete data analysis solution.

In the end, this project proves that data analytics plays a big role in the modern financial world. Raw data by itself does not mean much. But when it is cleaned, organized, analyzed, and visualized properly, it becomes very valuable. The insights gained from this project can help banks improve customer satisfaction, reduce risk, and increase their financial performance. Overall, this credit card financial analysis project successfully converts complex data into simple insights that can support better decision-making.

DECLARATION

I KAJAL RAWAL, a student pursuing Bachelor of Computer Application (**Data Analytics & Reporting**) at Amity University Online, hereby declare that the project work entitled "**Advanced Analytics for Credit Card Operations**" has been prepared by me during the academic year 2025 under the guidance of **Vaibhav Shah, AVP.** I assert that this project is a piece of original bona-fide work done by me. It is the outcome of my own effort and that it has not been submitted to any other university for the award of any degree.

Signature of Student

CERTIFICATE

This is to certify that **Kajal Rawal** of Amity University Online has carried out the project work presented in this project report entitled "**Advanced Analytics for Credit Card Operations**" for the award of Bachelor of Computer Application(**Data Analytics & Reporting**) under my guidance. The project report embodies results of original work, and studies are carried out by the student herself. Certified further, that to the best of my knowledge the work reported here in does not form the basis for the award of any other degree to the candidate or to anybody else from this or any other University/Institution.

Signature

Vaibhav Shah

(Name of Guide)

AVP

(Designation)

TABLE OF CONTENTS

I. Chapter 1: Introduction to the topic

II. Chapter 2: Review of Literature

III. Chapter 3: Research Objectives and Methodology

IV. Chapter 4: Data Analysis and Results

V. Chapter 5: Findings and Conclusion

VI. Chapter 6: Recommendations and Limitations of the study

VII. Bibliography / References

CHAPTER 1: INTRODUCTION TO THE TOPIC

In the modern financial world, data plays a very important role in shaping business decisions. With the rapid growth of digital banking and online payment systems, credit cards have become one of the most commonly used financial tools. People use credit cards for daily purchases, online shopping, travel, and emergency expenses. For banks and financial institutions, credit cards are not only a service offered to customers but also a major source of income through transaction fees, interest earned, and annual charges. As the usage of credit cards increases, a huge amount of data is generated every day in the form of transactions and customer records.

With the rapid growth of digital payments, financial organizations are increasingly relying on data-driven approaches to understand transaction patterns, monitor revenue streams, detect fraudulent activities, and evaluate customer usage trends. However, raw transaction data is often large, complex, and unstructured, making manual analysis inefficient and unreliable. To overcome these challenges, modern analytical tools such as SQL, Python, and Power BI are widely used to process, analyze, and visualize credit card transaction data in an effective and structured manner.

This project focuses on the analysis of credit card transaction data using **SQL for data extraction and database management**, **Python for data cleaning, transformation, and analytical processing**, and **Power BI for interactive data visualization and reporting**. SQL plays a crucial role in handling large datasets by enabling efficient querying, aggregation, and filtering of transaction records stored in relational databases. Python enhances the analytical process by supporting data preprocessing, statistical analysis, and pattern identification through powerful libraries such as Pandas and NumPy. Power BI complements these tools by

transforming analytical results into dynamic dashboards that provide meaningful insights through charts, KPIs, and visual summaries.

The integration of SQL, Python, and Power BI allows this project to follow an end-to-end data analytics workflow, starting from raw transaction data and ending with actionable business insights. Key performance indicators such as total revenue, transaction volume, customer spending behavior, card usage patterns, and interest earnings can be analyzed and presented in a visually intuitive manner. This approach not only improves data accuracy but also supports faster and more informed decision-making.

Overall, this credit card transaction report demonstrates how data analytics tools can be effectively combined to analyze financial data, identify trends, and support strategic planning in the banking and financial services sector. The project highlights the practical application of data analytics techniques and showcases the importance of transforming raw transaction data into meaningful insights using modern analytical technologies.

Analyzing this data is essential for understanding how customers use their credit cards, how revenue is generated, and where financial risks may arise. Credit card financial analysis helps institutions identify spending patterns, monitor credit utilization, detect delinquent accounts, and improve customer satisfaction. However, raw financial data is often unorganized, inconsistent, and difficult to analyze directly. It may contain missing values, duplicate records, incorrect formats, and extreme values that can affect the accuracy of analysis. Therefore, proper data cleaning, processing, and visualization are required before meaningful insights can be obtained.

This project focuses on performing a detailed credit card financial analysis using SQL, Python, and Power BI. These tools are widely used in the data analytics industry and together form a

complete analytical workflow. SQL is used to store, clean, and manage structured data efficiently. Python is used to further enhance and refine the data through flexible data manipulation and exploratory analysis. Power BI is used to present the final insights through interactive dashboards that are easy to understand and interpret.

The project uses two main datasets: a credit card transaction dataset and a customer dataset. The transaction dataset includes information such as card category, transaction amount, credit limit, interest earned, chip usage, and delinquency status. The customer dataset contains demographic details such as age, gender, income, education level, and customer satisfaction score. By combining these two datasets, the project provides a complete view of both financial performance and customer behavior.

Through this analysis, the project aims to understand key aspects such as revenue generation, transaction behavior, credit utilization patterns, and delinquency trends. The insights derived from this study can help financial institutions improve decision-making, reduce financial risk, and design better customer-focused strategies. Overall, this project demonstrates how data analytics can be effectively applied in the banking and financial sector to convert raw data into valuable business insights.

CHAPTER 2: LITERATURE REVIEW

A literature review helps us understand what other researchers, analysts, and industry experts have already done in the field of credit card analysis, financial analytics, customer behavior analysis, and data visualization. It also shows why the current project is important and how it fits into existing knowledge. This chapter summarizes previous studies and developments related to financial analytics, credit card usage patterns, data processing techniques, and dashboard-based decision-making.

Credit cards have become an essential part of everyday financial life across the world. From small daily purchases to large online transactions, credit cards offer convenience, flexibility, and short-term access to credit. Over time, they have evolved from simple payment instruments into complex financial products supported by advanced banking systems, data analytics, and regulatory frameworks. Several authoritative books have explored credit cards from economic, banking, consumer behavior, and risk management perspectives. This literature review brings together insights from these books to provide a clear understanding of how credit cards function, how risks are managed, and how customer behavior influences credit card systems.

Basic Concept of Credit Cards

A clear explanation of credit cards is provided in **The Economics of Consumer Credit**. The authors describe credit cards as revolving credit instruments that allow consumers to borrow money repeatedly up to a fixed credit limit. Unlike personal loans, credit cards do not require full repayment every month, which makes them highly flexible for users.

This flexibility is both an advantage and a risk. While consumers benefit from convenience and emergency funding, financial institutions face higher chances of delayed payments and defaults. The literature highlights how interest charges, annual fees, and penalties form the core revenue sources for banks issuing credit cards.

1. Credit Cards from a Banking Perspective

The operational side of credit cards is discussed in detail in **Bank Management & Financial Services**. This explains how credit cards fit into the overall retail banking system. Banks treat credit cards as high-revenue products but also recognize them as high-risk assets.

Banks carefully design credit card products by considering factors such as credit limits, interest rates, reward programs, and eligibility criteria. The literature emphasizes that banks rely heavily on customer data to decide who qualifies for a card and how much credit should be extended. Effective portfolio management helps bank balance profitability while controlling losses.

2. Credit Risk and Default Behavior

One of the most important challenges in credit card management is credit risk. This topic is strongly addressed in **Credit Risk Analytics**. The book explains that credit card risk is dynamic because customer spending and repayment behavior change frequently.

The literature discusses how banks use credit scores and behavioral models to predict the likelihood of default. Variables such as payment delays, credit utilization ratio, and transaction patterns are used to assess risk. Unlike traditional loans, credit cards require continuous monitoring, making analytics a critical part of credit card management.

3. Role of Data and Analytics in Credit Cards

Modern credit card systems generate massive volumes of data. This aspect is well explained in **Data Analytics for Finance**. Every transaction provides valuable insights into customer behavior, spending habits, and financial stability.

The literature highlights how banks use data analytics to segment customers, identify profitable users, and design personalized offers. Analytics also supports revenue forecasting and helps institutions understand trends such as seasonal spending and customer churn. Credit cards are presented as one of the richest data sources in the financial sector.

4. Fraud Detection and Security

Credit card fraud remains a major concern for financial institutions and consumers alike.

Traditional rule-based systems are no longer sufficient to handle modern fraud scenarios. Instead, banks use advanced analytical and machine learning techniques to identify suspicious transactions in real time. Protecting customers while ensuring smooth transactions is described as a key challenge in credit card security.

5. Consumer Behavior and Credit Card Usage

Consumer behavior plays a crucial role in the success of credit card products. This perspective is discussed in **Consumer Finance**. Factors such as convenience, reward programs, and promotional offers encourage increased credit card usage.

Highlights the psychological impact of credit cards, noting that consumers tend to spend more when using cards compared to cash. While this benefits businesses and banks, it can also lead to

overspending and debt accumulation. Understanding consumer behavior helps banks design responsible lending strategies.

6. Regulation and Ethical Considerations

Credit card systems operate under strict regulatory oversight. Governments and regulatory bodies impose rules to protect consumers from unfair practices.

Transparency in interest rates, fees, and repayment terms. Ethical credit card management includes responsible credit allocation and fair treatment of delinquent customers. These regulatory measures aim to maintain trust in the financial system while promoting financial inclusion.

7. Credit Cards in the Digital Era

The digital transformation of banking has significantly influenced credit card usage. **Digital Banking** discusses how credit cards have adapted to online payments, mobile wallets, and contactless technologies.

Instead of becoming obsolete, credit cards have become more integrated into digital platforms. Real-time transaction monitoring, digital onboarding, and personalized offers have improved customer experience and operational efficiency.

Credit card analytics has been a growing area of interest for financial institutions because customer spending behavior directly influences revenue, credit risk, and overall profitability. Many researchers have highlighted how transaction data can reveal strong patterns about customer lifestyle, preferences, and repayment habits. Studies show that analyzing customer

demographics—such as age, income, education level, and marital status—helps banks design better products and target the right market segments. This supports the idea that structured customer data, like the one used in this project, is important for predicting financial behavior and creating data-driven strategies.

Introduction to SQL

1. Introduction to SQL and Relational Databases

Structured Query Language (SQL) has been the backbone of relational database management systems (RDBMS) for several decades. According to SQL Notes for Professionals, SQL is a standardized, declarative programming language designed to manage, manipulate, and retrieve structured data stored in relational databases.

Since its introduction in the mid-1980s, SQL has evolved through multiple standards such as SQL-92, SQL:1999, SQL:2003, and SQL:2016, each adding more expressive power and advanced data handling features.

SQL is not limited to a single database system but acts as a universal interface across multiple platforms including MySQL, PostgreSQL, Oracle, SQL Server, and SQLite. This portability makes SQL an essential skill for data professionals, analysts, and developers. SQL's division into three major sub-languages: Data Definition Language (DDL), Data Manipulation Language (DML), and Data Control Language (DCL), which together enable comprehensive data management capabilities.

2. SQL Syntax, Identifiers, and Data Types

The proper use of identifiers and data types. Identifiers define names for database objects such as tables, columns, views, and indexes. The literature explains that identifiers may be quoted or unquoted, with behavior varying across database implementations. For example, case sensitivity differs between MySQL, PostgreSQL, Oracle, and SQL Server, which has implications for cross-platform compatibility and query portability.

Review of SQL data types, categorizing them into numeric, character, binary, date-time, and special types such as UNIQUEIDENTIFIER. Numeric types like INT, DECIMAL, FLOAT, and MONEY are discussed with respect to precision, storage requirements, and suitability for financial applications. The literature stresses the importance of selecting appropriate data types to ensure accuracy, performance, and efficient storage, particularly in large-scale database.

3. Handling NULL Values in SQL

One of the most conceptually challenging aspects of SQL highlighted in the book is the handling of NULL values. NULL represents the absence of a value and is fundamentally different from zero or an empty string. The literature emphasizes that incorrect handling of NULLs can lead to inaccurate query results and flawed business logic.

SQL's three-valued logic—TRUE, FALSE, and UNKNOWN—and how this affects filtering operations in WHERE clauses. It clearly demonstrates why equality comparisons with NULL are invalid and why IS NULL and IS NOT NULL must be used instead. This discussion is crucial for analytical queries, reporting, and data validation tasks.

4. Data Retrieval Using SELECT Statements

The SELECT statement is identified as the core of SQL querying. SELECT syntax, column selection, aliases, filtering conditions, sorting, and result limiting techniques. Excessive use of the wildcard character (*) in production environments due to performance and maintainability concerns.

Column aliases are presented as a best practice for improving query readability and supporting computed fields in views and reports. The literature also explains vendor-specific approaches to limiting query results, such as LIMIT, TOP, FETCH FIRST, and ROWNUM, highlighting the need for database-specific adaptations when writing portable SQL code.

5. Conditional Logic and Filtering in SQL

Significant attention to filtering mechanisms such as WHERE, HAVING, IN, BETWEEN, LIKE, and EXISTS clauses. These constructs allow SQL to express complex business rules directly within queries. The CASE statement is particularly emphasized as a powerful tool for implementing conditional logic within SELECT, ORDER BY, and UPDATE statements.

The literature illustrates how CASE expressions can transform raw data into meaningful categories, making SQL suitable for analytical reporting and dashboard preparation. This capability reduces dependency on external programming languages for data transformation.

6. Aggregation, GROUP BY, and Analytical Queries

Aggregation is a defining strength of SQL. Overview of aggregate functions such as COUNT, SUM, AVG, MIN, and MAX, explaining their role in summarizing large datasets. GROUP BY is discussed as a mechanism to partition data into meaningful subsets, enabling departmental, regional, or categorical analysis.

Advanced aggregation techniques such as ROLLUP and CUBE are introduced for multidimensional analysis, supporting decision-making processes in business intelligence and data mining contexts. The literature highlights the distinction between WHERE and HAVING clauses, reinforcing correct query design principles.

7. Joins and Relational Data Modeling

A substantial portion of JOIN operations, reflecting their importance in relational databases. INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, CROSS JOIN, SELF JOIN, and recursive joins are all discussed with practical examples. The literature explains how joins allow normalized tables to be recombined for meaningful analysis while preserving data integrity.

APPLY operators and correlated subqueries, expanding SQL's expressive power for advanced querying scenarios. These concepts are essential for handling hierarchical data and complex reporting requirements.

8. Subqueries, CTEs, and Window Functions

Modern SQL extends beyond simple querying to advanced analytical processing. The literature reviews subqueries in SELECT, FROM, and WHERE clauses, including correlated subqueries. Common Table Expressions (CTEs) are presented as a cleaner, more readable alternative to nested subqueries, particularly for recursive operations and temporary result sets.

Window functions such as ROW_NUMBER, LAG, LEAD, RANK, and running totals are highlighted as transformative features that enable SQL to perform sophisticated analytics traditionally reserved for statistical tools. These functions support time-series analysis, trend detection, and ranking operations.

9. Data Modification and Schema Management

Beyond querying, SQL provides robust mechanisms for data modification. INSERT, UPDATE, DELETE, MERGE, and TRUNCATE statements are discussed with emphasis on transactional safety and data consistency. Schema-level operations such as CREATE TABLE, ALTER TABLE, DROP TABLE, and constraint management.

Primary keys, foreign keys, indexes, and sequences are reviewed as essential components of well-designed databases. The literature stresses that thoughtful schema design improves query performance, ensures data integrity, and supports scalability.

10. Transactions, Security, and Best Practices

Transactional control using BEGIN, COMMIT, and ROLLBACK is identified as critical for maintaining data reliability. ACID principles and demonstrates how transactions protect against partial updates and system failures.

Improper query construction can expose databases to injection attacks, emphasizing parameterized queries and least-privilege access as best practices.

11. Clean Code and SQL Optimization

Execution order of SQL clauses is explained to help developers understand query behavior and improve performance.

This emphasis on clean code aligns SQL development with broader software engineering principles, reinforcing SQL’s role as a mature programming language rather than a simple querying tool.

Introduction to Python

1. Introduction to Python in Data Analytics

Python has emerged as one of the most influential programming languages in the field of data analytics due to its simplicity, flexibility, and extensive ecosystem of libraries. According to *Data Analytics with Python*, Python is a high-level, interpreted language that emphasizes readability and ease of learning, making it accessible to beginners while remaining powerful enough for advanced analytical tasks. The literature highlights that Python's design philosophy encourages rapid development, enabling analysts to focus more on problem-solving and insights rather than complex syntax.

Python as a unifying platform for the entire data analytics lifecycle, including data collection, cleaning, transformation, analysis, visualization, and interpretation. Its open-source nature and strong community support have further accelerated its adoption across academia, industry, and research domains.

2. Python Programming Fundamentals

A significant portion of the literature focuses on Python fundamentals, which form the foundation for analytical programming. Python's dynamic typing, indentation-based syntax, and object-oriented nature, emphasizing how these features contribute to cleaner and more maintainable code.

Core concepts such as variables, data types, operators, conditional statements, loops, and functions are explained as essential building blocks. Python's built-in data structures—lists, tuples, sets, and dictionaries—are particularly useful for handling real-world data scenarios.

These structures allow flexible data storage and efficient manipulation, which is critical in analytical workflows.

3. Role of Python in the Data Analytics Process

Python as an end-to-end solution for data analytics. It outlines a structured workflow beginning with data acquisition, followed by data cleaning, exploratory analysis, modeling, and visualization. Python's modular architecture enables seamless integration of multiple libraries at each stage of this process.

Python reduces dependency on multiple tools by offering a unified environment where data analysts can perform tasks that traditionally required separate software for statistics, visualization, and reporting.

4. NumPy for Numerical Computing

NumPy is identified as the foundation of numerical computation in Python. The book explains that NumPy provides support for multi-dimensional arrays, mathematical functions, and linear algebra operations that are optimized for performance.

NumPy arrays over traditional Python lists due to their speed and memory efficiency. Vectorized operations, broadcasting, and array indexing are discussed as features that enable fast computation on large datasets. These capabilities make NumPy essential for statistical analysis and scientific computing.

5. Pandas for Data Manipulation and Analysis

Pandas is described as the most important library for data analytics in Python. Pandas' core data structures Series and DataFrame which provide tabular data representation similar to spreadsheets or relational databases.

Pandas' ability to handle missing values, perform data filtering, sorting, grouping, merging, and aggregation. Time-series analysis, data reshaping, and pivot tables are also highlighted as powerful features that make Pandas suitable for business analytics and financial analysis.

6. Data Cleaning and Preprocessing

Data cleaning is identified as one of the most time-consuming yet critical stages of data analytics. Common data quality issues such as missing values, duplicates, inconsistent formats, and outliers.

Python's Pandas and NumPy libraries are presented as effective tools for handling these issues through techniques like imputation, normalization, type conversion, and filtering. Accurate preprocessing directly impacts the reliability of analytical results.

7. Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is emphasized as a key analytical phase that helps uncover patterns, trends, and anomalies. EDA through descriptive statistics, correlation analysis, and visualization techniques.

EDA enables analysts to form hypotheses and understand underlying data structures before applying advanced models. Python's interactive environment allows rapid experimentation and iterative analysis, making it well-suited for exploratory tasks.

8. Data Visualization Using Python

Visualization plays a critical role in communicating insights. Visualization libraries such as Matplotlib and Seaborn, which enable the creation of line charts, bar graphs, histograms, scatter plots, and heatmaps.

Visual representation simplifies complex data and enhances decision-making. Python's visualization ecosystem allows customization and integration with analytical workflows, making it a preferred choice for reporting and dashboards.

9. Statistical Analysis with Python

Python's capabilities in statistical analysis, including measures of central tendency, dispersion, probability distributions, and hypothesis testing. Libraries such as NumPy and SciPy are discussed as tools for implementing statistical techniques.

Python allows analysts to move beyond descriptive statistics to inferential analysis, enabling data-driven conclusions and predictive insights.

10. Introduction to Machine Learning Concepts

Although primarily focused on analytics, basic machine learning concepts to demonstrate Python's extended capabilities. Topics such as regression, classification, and clustering are briefly discussed using Python-based tools.

Python's compatibility with machine learning libraries enables a smooth transition from traditional analytics to predictive modeling, making it future-ready for advanced data science applications.

11. Python for Business and Real-World Applications

Python's relevance in real-world business contexts such as finance, marketing, healthcare, and operations. Use cases include customer segmentation, sales forecasting, fraud detection, and performance analysis.

Python's flexibility allows it to adapt to domain-specific requirements while maintaining consistency in analytical methods.

12. Advantages of Python in Data Analytics

Several advantages of Python are identified in the literature, including ease of learning, extensive libraries, cross-platform compatibility, and strong community support. Python's open-source nature reduces cost barriers and encourages innovation.

Python integrates well with databases, big data frameworks, and visualization tools, further strengthening its position in the analytics ecosystem.

13. Limitations and Challenges

Despite its strengths, the literature acknowledges certain limitations of Python. Performance issues may arise with extremely large datasets compared to low-level languages. Additionally, dependency management and version compatibility can pose challenges.

However, limitations are often mitigated through optimized libraries and scalable frameworks.

Data visualization plays a critical role in data analysis by transforming numerical data into visual formats that are easy to understand and interpret. As datasets grow in size and complexity, visual

tools become essential for identifying trends, patterns, relationships, and anomalies. Among the various visualization libraries available in Python, Matplotlib is one of the most widely used and foundational tools. It provides extensive functionality for creating static, animated, and interactive visualizations, making it a core component of Python's data science ecosystem.

Matplotlib was originally developed by John D. Hunter and has since become a standard library for data visualization in Python. The library is designed to offer fine-grained control over plots while remaining flexible enough to support a wide range of visualization needs. This literature review discusses the evolution, features, applications, strengths, and limitations of Matplotlib as presented in books, documentation, and academic discussions.

Evolution and Purpose of Matplotlib

Matplotlib was created to provide Python users with a plotting library comparable to MATLAB's plotting capabilities. According to the official documentation of Matplotlib, the library was designed to generate publication-quality figures across different platforms. Early literature highlights that Matplotlib filled a significant gap in Python by enabling scientists and analysts to visualize data without relying on external tools.

Matplotlib forms the foundation upon which many other visualization libraries are built. Even modern libraries like Seaborn and Pandas plotting functions rely internally on Matplotlib. This reinforces its importance as a core visualization tool rather than just an optional add-on.

1. Core Features and Plotting Capabilities

Matplotlib supports a wide variety of plots, including line charts, bar graphs, histograms, scatter plots, pie charts, box plots, and heatmaps. Literature consistently highlights its flexibility in customizing plots, including control over colours, labels, legends, axes, annotations, and layouts. This level of customization allows users to tailor visualizations for academic publications, business dashboards, and exploratory analysis.

Another important feature discussed in literature is Matplotlib's object-oriented interface. While beginners often use the simple pyplot interface, advanced users prefer the object-oriented approach for greater control and scalability. This dual interface design makes Matplotlib suitable for both beginners and experienced analysts.

2. Matplotlib in Exploratory Data Analysis

Exploratory Data Analysis (EDA) is a crucial stage in any data analytics project, and Matplotlib plays a key role in this process. Literature emphasizes that visualization helps analysts understand data distribution, identify outliers, and examine relationships between variables before applying advanced models.

Histograms and box plots created using Matplotlib are especially useful for understanding data spread and skewness. Scatter plots help identify correlations, while line plots are effective for time-series analysis. These capabilities make Matplotlib an essential tool for initial data exploration.

3. Integration with Python Data Libraries

A major strength of Matplotlib highlighted in literature is its seamless integration with other Python libraries such as NumPy and Pandas. Since most analytical data in Python is stored in NumPy arrays or Pandas DataFrames, Matplotlib can directly visualize these structures without additional transformation.

This integration simplifies the analytical workflow and allows analysts to move smoothly from data processing to visualization. Literature frequently notes that this compatibility enhances productivity and reduces complexity in data analysis projects.

4. Use of Matplotlib in Academic and Business Applications

Matplotlib is widely used in academic research due to its ability to produce high-quality, publication-ready figures. Scientific papers, theses, and research reports often rely on Matplotlib for visual representation of experimental results and statistical findings.

Matplotlib is commonly used for internal analysis, reporting, and dashboard development. Although interactive tools like Power BI and Tableau are preferred for executive reporting, Matplotlib remains valuable for backend analysis and custom reporting where full control over visuals is required.

5. Customization and Aesthetic Control

Matplotlib’s strength in customization. Users can modify almost every element of a plot, including fonts, markers, gridlines, figure size, and resolution. This makes Matplotlib particularly suitable for situations where visual standards must be strictly followed, such as academic journals or corporate branding guidelines.

However, the same flexibility is also described as a challenge for beginners. Matplotlib requires more code compared to high-level libraries. Despite this, learning Matplotlib is considered essential because it provides a deeper understanding of visualization principles.

6. Limitations of Matplotlib

While Matplotlib is powerful, literature also discusses its limitations. One commonly cited drawback is its lack of native interactivity compared to modern visualization tools. Creating interactive dashboards requires additional libraries or frameworks.

Compared to libraries like Seaborn, Matplotlib often requires more lines of code to achieve the same visual result. Despite these limitations, literature emphasizes that Matplotlib remains indispensable due to its flexibility and foundational role.

7. Matplotlib as a Learning Tool

Matplotlib as the first visualization library for Python learners. Literature suggests that understanding Matplotlib helps learners grasp key visualization concepts such as axes, scales, and plot composition. Once these fundamentals are understood, transitioning to higher-level libraries becomes easier.

In the field of data analytics, efficient data handling and manipulation are critical for generating meaningful insights. Raw data collected from real-world sources is often incomplete, inconsistent, and unstructured, making analysis difficult without proper pre-processing tools. Python Pandas has emerged as one of the most powerful and widely used libraries for data

manipulation and analysis. It provides flexible data structures and high-level functions that simplify working with structured data such as tables, time-series, and transactional datasets.

Python Pandas was developed by Wes McKinney to address the limitations of existing tools for data analysis in Python. Since its introduction, Pandas has become a core component of the Python data ecosystem and is extensively used in academia, industry, and research. This literature review discusses the evolution, features, applications, advantages, and limitations of Pandas as presented in books, documentation, and analytical studies.

The development of Pandas was motivated by the need for a fast, flexible, and easy-to-use data analysis tool in Python. According to the official documentation of Pandas, the library was designed to make data cleaning, transformation, and analysis more intuitive and efficient.

Pandas, Python lacked a native structure comparable to spreadsheets or relational database tables. Pandas filled this gap by introducing data structures that closely resemble tabular data formats, enabling analysts to perform complex operations with minimal code. Pandas has significantly reduced the learning curve for data analysis in Python.

Pandas is built around two primary data structures: **Series** and **DataFrame**. A Series represents a one-dimensional labelled array, while a DataFrame is a two-dimensional labelled data structure similar to a table. Literature consistently highlights that these structures provide a strong foundation for data analysis by allowing easy indexing, slicing, and alignment of data.

DataFrames are particularly important in analytical projects because they allow analysts to work with rows and columns intuitively. Column-wise operations, row filtering, and label-based

indexing make Pandas suitable for handling datasets such as credit card transactions, sales records, and customer information.

One of the most widely discussed applications of Pandas is data cleaning and pre-processing. Real-world datasets often contain missing values, duplicates, inconsistent formats, and outliers. Pandas provides built-in functions to detect and handle these issues efficiently.

Functions such as isnull(), fillna(), dropna(), and duplicated() in preparing data for analysis. Type conversion, date-time handling, and string manipulation features further enhance Pandas' ability to prepare clean and reliable datasets. Effective pre-processing using Pandas directly improves the accuracy and quality of analytical results.

Exploratory Data Analysis (EDA) is a crucial step in understanding data before applying advanced analytical or predictive techniques. Pandas supports EDA by providing descriptive statistics, summary functions, and aggregation methods.

The use of functions such as describe(), groupby(), and value_counts() to examine data distributions and trends. These features allow analysts to identify patterns, detect anomalies, and generate hypotheses. Pandas' ability to quickly summarize large datasets makes it an essential tool for initial data exploration.

Pandas is widely used in business and financial analytics due to its ability to handle large transactional datasets. Pandas is particularly effective for analyzing financial records such as credit card transactions, revenue data, customer behaviour, and expense patterns.

Time-series analysis features in Pandas enable analysts to study trends over time, making it suitable for financial forecasting and performance evaluation. The ability to merge and join datasets also supports multi-source data analysis, which is common in real-world business scenarios.

Several advantages of Pandas are consistently highlighted in literature. These include ease of use, expressive syntax, flexibility, and powerful data manipulation capabilities. Pandas allows complex operations to be performed with relatively simple and readable code.

The open-source nature of Pandas and its strong community support further enhance its adoption. Continuous updates and extensive documentation make it accessible to both beginners and advanced users.

Pandas is frequently recommended as the first data analysis library for Python learners. Pandas helps users develop a strong understanding of data structures, indexing, and data manipulation concepts. These skills form a solid foundation for advanced analytics and machine learning.

Credit Card Revenue Sources and Risk Indicators

Credit card revenue mainly comes from annual fees, interest earned, and transaction volume. According to financial analytics studies, frequent spending, high credit limits, and consistent usage of chip-based transactions often indicate healthier customer engagement. Researchers also suggest that high credit utilization and delinquent accounts are strong indicators of financial risk. These findings support the KPIs used in this project, such as utilization ratio, interest earned, and delinquency tracking, which help identify both profitable and risky customers.

Several studies highlight the importance of data cleaning and preprocessing in financial datasets. Real-world transactional data is usually messy and inconsistent, which is why removing duplicates, correcting formats, and handling missing values are essential steps. Researchers emphasize that poor data quality can lead to incorrect conclusions, making SQL and Python crucial tools for preparing accurate datasets. This aligns with the methodology used in our project, where SQL and Python work together to ensure clean and reliable data before visualization.

In the area of data analysis tools, previous literature shows that SQL is one of the most popular methods for managing large financial datasets. SQL allows analysts to filter, group, and aggregate data efficiently, making it suitable for calculating financial metrics. Python is recognized for its powerful data processing libraries, which help automate cleaning tasks and perform statistical analysis. Many researchers appreciate that SQL and Python complement each other, which is exactly how they are used in this project.

POWER BI INTRODUCTION

Power BI has also gained wide recognition in academic and industry research. It is commonly used for business intelligence, financial reporting, and interactive dashboards. Studies show that visual analytics tools like Power BI help users understand large datasets quickly, even if they don’t have technical expertise. By presenting data through charts, graphs, and KPIs, Power BI makes financial data easier to interpret. Literature also points out that dashboards help managers make faster and more accurate decisions. This supports the use of Power BI in our project as the main visualization tool.

Advanced Analytics for Credit Card Operations

1. Business Intelligence Context

Business Intelligence (BI) refers to technologies and practices that transform raw data into meaningful insights for decision-making. Power BI sits within this domain as a comprehensive suite of tools that integrates data preparation, visualization, and collaboration. The course material underscores BI as a framework for descriptive and diagnostic analytics, where tools like Power BI allow organizations to monitor performance, identify trends, and support strategic planning.

Scholars argue that BI tools are critical in enabling data-driven decision-making frameworks (DDDM), reducing reliance on intuition and improving organizational efficiency. Power BI, in particular, democratizes BI by offering user-friendly interfaces and cloud-based accessibility, making advanced analytics available to non-technical users.

2. Components of Power BI

The core components of Power BI, each serving distinct functions in the analytics pipeline:

- **Power BI Desktop**: A Windows-based application for data modeling, visualization, and report creation. It supports multiple views, tabs, and integration with diverse data sources.
- **Power BI Service**: A cloud-based platform for sharing, collaboration, and enterprise deployment of dashboards.
- **Power BI Mobile**: Extends accessibility to smartphones and tablets, enabling decision-makers to access insights on the go.
- **Power BI Gateway**: Facilitates secure data transfer between on-premises sources and cloud services.

- **Power BI Embedded**: Allows developers to integrate BI capabilities into custom applications.

3. Features and Advantages

Power BI's literature consistently highlights its distinctive features:

- **Data Connectivity**: Supports a wide range of data sources, including Excel, SQL databases, cloud services, and APIs.
- **Visualization Capabilities**: Offers customizable charts, maps, and dashboards, enabling effective storytelling with data.
- **Ease of Use**: Designed with intuitive drag-and-drop interfaces, lowering the barrier for non-technical users.
- **Collaboration**: Cloud-based sharing and integration with Microsoft Teams enhance organizational communication.
- **Scalability**: Suitable for small businesses and large enterprises alike, with flexible licensing models.

Business Intelligence (BI) tools have become essential for organizations seeking to transform raw data into meaningful insights. Among these tools, Power BI has gained significant popularity due to its ability to connect data, perform analysis, and present insights through interactive dashboards. A key component that enables advanced analytics within Power BI is DAX (Data Analysis Expressions). DAX is a formula language used to create calculated columns, measures, and tables, allowing users to perform complex calculations on data models.

Literature on business intelligence and data analytics identifies DAX as a powerful yet specialized language that bridges the gap between raw data and analytical reporting. This review discusses the evolution, purpose, core concepts, applications, strengths, and limitations of DAX as described in BI-focused books, documentation, and analytical studies.

4. DAX and the Tabular Data Model

A central concept emphasized in the literature is the relationship between DAX and the tabular data model. DAX operates on data stored in tables that are connected through relationships, similar to a relational database. The evaluation of DAX expressions depends heavily on row context and filter context, which are fundamental concepts repeatedly discussed in BI literature.

Understanding context is considered essential for mastering DAX. Filter context determines which subset of data is considered during a calculation, while row context refers to the current row being evaluated. This context-based evaluation makes DAX extremely powerful but also challenging for beginners.

5. Calculated Columns and Measures

DAX is mainly used to create calculated columns and measures. Calculated columns generate values at the row level and are stored in the data model, whereas measures are calculated dynamically based on the filters applied in reports. Measures are preferred for most analytical scenarios due to their flexibility and performance efficiency.

In financial and transaction-based analysis, measures are widely used to calculate metrics such as total revenue, average transaction value, interest earned, and utilization ratios. These dynamic

calculations allow analysts to explore data interactively, which is a key advantage of using DAX in Power BI.

6. Common DAX Functions and Expressions

The DAX as a rich language with a wide range of functions categorized into aggregation, logical, mathematical, time intelligence, and filter functions. Functions such as SUM, AVERAGE, CALCULATE, and FILTER are frequently cited as core building blocks for most DAX expressions.

DAX enables calculations such as year-to-date, month-to-date, and year-over-year comparisons, which are essential for trend analysis and performance tracking.

7. Role of DAX in Business and Financial Analytics

The role of DAX in business and financial analytics. In projects involving sales, revenue, or credit card transactions, DAX allows analysts to define custom KPIs and financial indicators that cannot be derived directly from raw data.

For example, in credit card transaction analysis, DAX can be used to calculate total revenue by combining annual fees, transaction amounts, and interest earned. It also supports segmentation analysis based on card category, customer type, and transaction channel. This flexibility makes DAX a critical tool for decision-making dashboards.

8. Advantages of Using DAX

Several advantages of DAX are emphasized in the literature. One major strength is its ability to perform complex calculations on large datasets with high performance due to Power BI's in-

memory engine. DAX also supports reusability, allowing measures to be used across multiple visuals and reports.

Another advantage is its tight integration with Power BI visuals. DAX calculations respond dynamically to filters and slicers, enabling interactive data exploration. Literature also notes that DAX promotes consistency in reporting by centralizing business logic within the data model.

9. Challenges and Limitations of DAX

Despite its strengths, DAX has a steep learning curve. The concepts of context and evaluation order can be difficult for beginners to grasp. Unlike procedural languages, DAX does not follow a step-by-step execution flow, which can make debugging challenging.

Another limitation discussed in complex DAX expressions can impact performance if not optimized properly. Poorly designed measures may lead to slow report rendering, especially with large datasets. However, these challenges can be addressed through best practices and model optimization.

10. DAX as a Skill in Modern Analytics

Many BI-focused books and academic sources emphasize that DAX is an essential skill for Power BI professionals. While basic reports can be created without DAX, advanced analytics and meaningful insights rely heavily on well-written DAX measures.

Literature suggests that learning DAX enhances an analyst’s ability to translate business requirements into analytical logic, making it a valuable skill in modern data-driven organizations.

Many previous works have focused on identifying customer behavior patterns through credit card transactions. These studies show that spending categories—such as shopping, travel, groceries, fuel, and entertainment—reveal lifestyle preferences and financial habits. Expense-type analysis helps banks design personalized offers and loyalty programs. Similarly, researchers have found that customer satisfaction scores are important for measuring service quality and predicting long-term customer relationships. The customer table in our project includes satisfaction scores for this reason, allowing deeper insight into the customer experience.

Studies also highlight that combining demographic data with transaction data provides a more complete understanding of customer behavior. For example, younger customers may use their credit cards more frequently for online shopping, while older customers may use them for bill payments and household expenses. Higher-income customers may have higher credit limits and more frequent transactions. Literature supports the idea that combining both tables—credit_card and customer—can reveal stronger insights, just like in this project.

Another important topic in past research is risk analysis. Delinquency is a key area of concern for banks because unpaid balances can create financial losses. Researchers have found that customers with high utilization ratios or inconsistent payment behavior are more likely to become delinquent. Studies suggest that monitoring delinquent accounts can help banks take preventive actions.

In summary, the literature reviewed in this chapter shows that credit card analytics is a well-researched and valuable area for financial institutions. Previous studies support the use of demographic data, transaction data, utilization ratios, and spending types to understand customer behavior and financial performance. They also highlight the importance of data cleaning,

structured databases, and visualization tools. The combination of SQL, Python, and Power BI used in our project aligns with best practices identified in academic and industry research. This literature review shows that the current project builds on strong foundations and contributes by creating a complete, practical system for credit card financial analysis.

.

.

CHAPTER 3. RESEARCH OBJECTIVES AND METHODOLOGY

Objectives of the Study

The main objective of this project is to analyze credit card financial data and extract useful insights that help in understanding customer behavior and financial performance. To achieve this main goal, several specific objectives have been defined.

One of the primary objectives is to clean and prepare the raw data using SQL. This involves identifying and handling missing values, removing duplicate records, correcting incorrect text formats, and ensuring consistency across all fields. Clean and reliable data is the foundation of accurate analysis, and this step ensures that errors in the dataset do not affect the final results.

Another important objective is to analyze revenue generated from credit card usage. This includes studying components such as transaction amounts, interest earned, and annual fees. By examining these elements, the project aims to identify which factors contribute most to overall revenue and how different card categories and customer segments influence profitability.

Understanding customer spending behavior is also a key objective of this study. The project analyzes transaction frequency, total transaction amount, and credit utilization ratio to understand how customers use their credit cards. These insights help in identifying high-value customers as well as customers who may be at higher financial risk.

The project also aims to study delinquency patterns among customers. Delinquency occurs when customers fail to make payments on time, which can lead to financial losses for institutions. By analyzing delinquent accounts in relation to income level, credit limit, and utilization ratio, the

project seeks to identify trends that may indicate higher risk. This objective supports better credit risk management and policy formulation.

Another objective is to enhance and refine the data using Python. Python allows for advanced data processing, validation of SQL results, and creation of new derived variables. Using Python helps in improving data quality and enables deeper exploratory analysis that may not be easily achieved using SQL alone.

Finally, an important objective of this project is to design interactive dashboards using Power BI. These dashboards present insights in a visual and user-friendly manner, making it easier for decision-makers to understand trends and patterns. The goal is not only to analyze data but also to communicate insights effectively through visuals.

Scope and Limitations of the Study

Scope of the Study

The scope of this project is focused on the comprehensive analysis of credit card transaction and customer-related data using **SQL, Python, and Power BI**. The study emphasizes **descriptive and diagnostic analytics** to examine historical patterns and relationships within the dataset. Key areas of analysis include revenue trends, transaction behavior, credit utilization levels, and delinquency patterns to assess overall credit card financial performance.

In addition, the project incorporates **demographic analysis** based on customer attributes such as age, gender, income level, and customer satisfaction scores. This analysis supports effective customer segmentation and helps identify behavioral differences across demographic groups.

The project also covers essential stages of the data analytics lifecycle, including **data cleaning, data enhancement, exploratory data analysis (EDA), and interactive dashboard creation**.

By integrating SQL for data extraction and transformation, Python for analysis and visualization, and Power BI for dashboard development, the project delivers an **end-to-end analytical workflow** that closely reflects real-world data analytics practices used in financial services organizations. The insights generated from this study are intended to support data-driven decision-making related to credit card performance and customer behavior.

Limitations of the Study

Despite its practical value, this study has several limitations that should be acknowledged. First, the analysis is based entirely on **historical data** provided within the dataset and does not include **real-time or live transaction data**. As a result, the findings reflect past trends and behaviors and may not capture recent changes in customer spending patterns or financial conditions.

Another significant limitation is the **absence of predictive or machine learning models**. The project focuses solely on understanding existing patterns rather than forecasting future outcomes. Advanced analytical techniques such as **credit risk prediction, customer churn modeling, or fraud detection** are beyond the scope of this study.

Furthermore, the dataset does not include **external or supplementary data sources**. Important influencing factors such as macroeconomic conditions, credit history from other financial institutions, lifestyle behaviors, and regional economic variations are not considered. The absence of these factors may limit the depth of insights related to spending behavior and delinquency risk.

METHODOLOGY

The methodology of this project explains the systematic approach followed to analyze credit card financial data using SQL, Python, and Power BI. This chapter outlines the data sources, tools used, data preparation steps, analytical techniques applied, and the process followed to generate meaningful insights. The methodology was designed to ensure accuracy, consistency, and clarity throughout the analysis, while closely reflecting real-world data analytics practices used in the banking and financial sector.

1. Data Collection and Understanding

The first step of the methodology involved understanding the nature of the data used in the project. The dataset consists of two primary tables: a **credit card transaction table** and a **customer detail table**. The credit card table contains transaction-related and financial attributes such as card category, transaction amount, credit limit, interest earned, utilization ratio, chip usage, and delinquency status. The customer table includes demographic and personal information such as age, gender, income, education level, marital status, and customer satisfaction score.

Before starting the analysis, the structure of each table and the meaning of every column were carefully studied. Understanding how the transaction data relates to customer data was essential to ensure proper joins and meaningful analysis. The common key used for linking both tables was the client number, which uniquely identifies each customer.

2. Tools and Technologies Used

This project uses a combination of SQL, Python, and Power BI, each serving a specific purpose in the data analytics lifecycle.

- **SQL** was used for data storage, data cleaning, data transformation, and basic analysis. It allowed efficient handling of structured data and execution of queries for filtering, aggregating, and joining tables.
- **Python** was used for advanced data cleaning, enhancement, and exploratory data analysis. Libraries such as Pandas and NumPy were used to manipulate datasets, validate results, and create additional derived features.
- **Power BI** was used for data visualization and dashboard creation. It helped convert complex data into interactive and easy-to-understand visual insights for decision-making.

The integration of these three tools ensured an end-to-end analytical workflow from raw data to final insights.

3. Data Cleaning Using SQL

Data cleaning was a critical step in the methodology, as raw financial data often contains inconsistencies and errors. SQL was used as the primary tool for initial data cleaning.

The following data cleaning activities were performed:

- **Handling missing values:** Records with missing or NULL values in important fields such as transaction amount, credit limit, and income were identified. Depending on the importance of the field, missing values were either replaced with appropriate defaults or excluded from analysis.

- **Removing duplicate records:** Duplicate entries were checked using client number and transaction details to ensure that each record represented unique information.

- **Correcting incorrect data formats:** Text fields such as chip usage and expense type often contained inconsistent formats (for example, “Use chip” and “use Chip”). These values were standardized using SQL update queries.

- **Validating numerical values:** Negative or unrealistic values in transaction amount, credit limit, and age were identified and corrected or removed.

- **Outlier detection:** Extremely high or low values in transaction amount and credit limit were examined to ensure they did not distort the analysis.

These steps ensured that the dataset was accurate, consistent, and reliable before moving to advanced analysis.

4. Data Transformation and Feature Creation

After cleaning the data, SQL was used to transform the dataset and create derived fields that added analytical value. New calculated columns such as total revenue were created by combining transaction amount, interest earned, and annual fees. Credit utilization ratios were analyzed to understand how much of the credit limit customers were using.

Categorical fields such as age groups and income brackets were created to support segmentation analysis. These transformations helped simplify analysis and improved the clarity of insights.

5. Data Enhancement and Refinement Using Python

Once the cleaned data was extracted from the database, Python was used for further enhancement and refinement. Python provided greater flexibility for data manipulation and exploratory analysis.

Using Python:

- The dataset was revalidated to ensure consistency with SQL results.
- Additional data cleaning steps were performed, such as handling remaining missing values and standardizing categorical variables.
- Feature engineering was applied to create meaningful indicators such as customer risk levels and spending categories.
- Aggregations and group-based analysis were performed to identify trends across customer segments.

Python also allowed quick experimentation and visualization during exploratory analysis, helping to identify patterns that guided dashboard design.

6. Exploratory Data Analysis (EDA)

Exploratory Data Analysis was conducted using both SQL and Python to understand data distribution and relationships between variables. Key metrics such as transaction amount, transaction count, credit utilization, and delinquency rates were analyzed across different customer segments.

EDA helped answer important questions such as:

- Which card categories generate the highest revenue?
- How does income level affect spending behavior?
- What is the relationship between credit utilization and delinquency?
- Which customer segments contribute most to revenue?

This step played a vital role in shaping the final dashboards and insights.

7. Data Visualization Using Power BI

After data preparation and analysis, the refined dataset was imported into Power BI for visualization. Power BI was chosen due to its strong interactive capabilities and ease of use.

The dashboards were designed to display:

- Key Performance Indicators (KPIs) such as total revenue, total transactions, and average utilization.
- Revenue breakdown by card category, expense type, and transaction method.
- Customer segmentation based on age, income, and satisfaction score.
- Delinquency trends and risk indicators.

Filters and slicers were added to allow users to interact with the data and explore specific segments easily. The focus was on clarity, simplicity, and business relevance.

8. Validation and Accuracy Check

To ensure accuracy, results obtained from SQL queries were cross-checked using Python analysis. Any inconsistencies were reviewed and corrected. Dashboard outputs were validated by comparing them with aggregated SQL results. This step ensured that insights presented were reliable and trustworthy.

9. Ethical Considerations and Data Limitations

Throughout the methodology, data privacy and ethical considerations were maintained. The dataset contained no personally identifiable information such as names or contact details. The analysis was performed strictly for academic purposes.

The methodology is limited to descriptive and diagnostic analysis based on historical data. Predictive modeling and real-time analysis were not included in this project.

10. Summary of Methodology

In summary, the methodology followed a structured and logical approach, starting from data understanding and cleaning to advanced analysis and visualization. The combined use of SQL, Python, and Power BI ensured a professional, industry-relevant workflow. This methodology supports accurate insights and effective communication of results and provides a strong foundation for future analytical enhancements.

CHAPTER 4. DATA ANALYSIS, RESULTS, AND INTERPRETATION

4.1 DATA ANALYSIS

The system architecture explains how different parts of the project work together to create the final Credit Card Financial Analysis system. It shows how data flows from its raw form to the final interactive dashboard. The architecture is designed to be simple, efficient, and easy to understand. It uses three main technologies—SQL, Python, and Power BI—which together form a complete end-to-end analytics solution.

At the core of the system is the database layer, where all credit card and customer data are stored in structured SQL tables. The database contains two main tables: credit_card and customer. The credit_card table holds information about transactions, fees, credit limits, utilization ratio, and revenue-related fields. The customer table stores demographic details such as age, income, gender, education, and satisfaction score. Storing data in a relational database ensures it is organized, consistent, and easy to query. SQL also allows fast data retrieval, which is necessary for calculations and cleaning.

The next part of the architecture is the data processing layer, where SQL and Python work together. SQL is used first for cleaning, filtering, and transforming the data. This includes formatting dates, correcting text values, removing duplicates, and generating basic metrics. After this, Python is used for deeper cleaning and exploration. Python helps handle missing values, identify outliers, and prepare the data for visualization. The combination of SQL and Python makes the data more accurate and complete.

Once the data is processed and ready, it moves to the visualization layer, which is built in Power BI. Power BI connects directly to the cleaned dataset and creates the final dashboards. It uses visuals like bar charts, line graphs, pie charts, KPIs, and slicers. The dashboard allows users to filter by card category, gender, age group, income level, and expense type. This layer gives the final, user-friendly view of the project.

The system architecture can be summarized in four simple steps:

Data analysis plays a vital role in transforming raw credit card transaction data into meaningful financial insights. In this project, data analysis was carried out using a combination of SQL, Python, and Power BI to ensure accuracy, efficiency, and effective visualization. The analytical process involved data extraction, cleaning, transformation, aggregation, and visualization, allowing a comprehensive understanding of credit card financial performance and customer behavior.

The analysis focuses on evaluating transaction trends, revenue components, card usage patterns, and expense distribution. By integrating database queries, analytical programming, and business intelligence tools, this project demonstrates a complete end-to-end data analysis workflow.

4.1 Data Analysis

Data Extraction and Preparation Using SQL

SQL was used as the primary tool for extracting and organizing credit card transaction data stored in a relational database. The dataset contained information such as customer details, card category, transaction amount, interest earned, annual fees, expense type, transaction mode, and time-related attributes.

Using SQL queries, data was filtered, grouped, and aggregated to calculate key financial metrics. Operations such as SUM, COUNT, AVG, and GROUP BY were used to generate summaries including total transaction amount, total interest earned, total revenue, and transaction volume. SQL also helped identify duplicate records and validate data consistency before further analysis.

Data Cleaning and Transformation Using Python

After extracting data from the database, Python was used for data cleaning and transformation. Libraries such as Pandas and NumPy were applied to handle missing values, remove duplicates, and standardize data formats. Date columns were converted into proper date formats to support time-based analysis.

Python enabled deeper analytical processing by creating derived columns such as average transaction value and utilization ratios. Exploratory Data Analysis (EDA) was performed to understand data distribution, detect outliers, and identify unusual patterns. Python's flexibility allowed efficient handling of large datasets and preparation of clean data for visualization.

Revenue Analysis

Revenue analysis was a major focus of this project. Revenue was calculated by combining key financial components such as annual fees, transaction amounts, and interest earned. The analysis revealed which revenue streams contributed the most to overall financial performance.

Transaction-based revenue was found to be the largest contributor, followed by interest income and annual fees. Python and SQL were used together to compute revenue trends across different

card categories and time periods. This analysis provided insights into how customer usage and repayment behavior impact overall revenue.

Transaction Volume and Trend Analysis

Transaction volume analysis was conducted to understand customer activity and usage frequency. SQL queries were used to calculate the number of transactions across different card types and expense categories. Python was then used to analyze trends over time, identifying peak transaction periods and seasonal patterns.

The analysis showed variations in transaction volume based on card category and usage mode. Certain card types demonstrated higher transaction frequency, indicating greater customer engagement. These insights help financial institutions evaluate product performance and customer preferences.

Expense Type Analysis

Expense type analysis was performed to understand how customers spend using credit cards. Transactions were categorized into different expense types, such as shopping, travel, groceries, and utilities. SQL was used to aggregate transaction amounts by expense category, while Python supported comparative analysis.

The results indicated that a few expense categories dominated total spending, reflecting common consumer behavior patterns. Understanding expense distribution helps banks design targeted offers, reward programs, and spending-based incentives.

Advanced Analytics for Credit Card Operations

Card Usage Pattern Analysis

Card usage patterns were analyzed based on transaction modes such as chip-based payments, online transactions, and swipe transactions. Python and SQL were used to compare transaction volume and revenue across different usage modes.

The analysis revealed that chip-based and online transactions contributed significantly to total revenue. These insights highlight the growing importance of digital and secure payment methods in the credit card ecosystem.

Customer Behavior Analysis

Customer behavior analysis focused on spending levels, transaction frequency, and repayment-related indicators. Python was used to group customers based on transaction behavior and identify high-value customers.

The analysis showed that a small segment of customers contributed a significant portion of total revenue, aligning with common financial distribution patterns. These findings are useful for customer segmentation and personalized financial product development.

Visualization and Reporting Using Power BI

Power BI was used to convert analytical results into interactive dashboards. Key performance indicators such as total revenue, total transactions, interest earned, and average transaction value were displayed using KPI cards.

Visualizations such as bar charts, line graphs, and pie charts were used to present revenue distribution, transaction trends, and expense breakdowns. Filters and slicers allowed dynamic analysis by card category, expense type, and time period. Power BI enhanced data storytelling and made insights accessible to non-technical users.

Integration of SQL, Python, and Power BI

A major strength of this project lies in the integration of SQL, Python, and Power BI into a unified analytical workflow. SQL ensured efficient data retrieval, Python enabled advanced analysis and data preparation, and Power BI provided powerful visualization capabilities.

This integrated approach improved data accuracy, reduced manual effort, and supported real-time analytical exploration. It also demonstrates how modern analytics tools can be combined to solve real-world financial problems.

4.2 DATA PREPARATION

Data preparation is one of the most important steps in the entire project because the quality of insights depends on the quality of data. In this project, the data preparation process was carried out using both **SQL** and **Python**. The goal was to clean, organize, and transform the raw credit card dataset into a reliable and analysis-ready format. This chapter explains how the data was prepared in a step-by-step, easy-to-understand manner.

Understanding the Raw Data

The raw dataset contained credit card transaction details, customer information, spending behavior, revenue-related fields, and several demographic attributes.

Before cleaning, the dataset had issues such as:

Figure 4.2.1: Missing Values Distribution

```sql
-- Check for NULL Values
SELECT *
FROM credit_card
WHERE Client_Num IS NULL
   OR Card_Category IS NULL
   OR Total_Trans_Amt IS NULL
   OR Interest_Earned IS NULL;

-- Replace NULL Values
UPDATE credit_card
SET Interest_Earned = 0
WHERE Interest_Earned IS NULL;
```

This figure highlights the count of missing entries across different variables in the dataset, helping to identify columns with significant data gaps that may require imputation, removal, or further investigation.

Figure 4.2.2: Inconsistent date formats

```sql
-- Fix Date Format
UPDATE credit_card
SET Week_Start_Date = STR_TO_DATE(Week_Start_Date, '%m/%d/%Y');
```

This figure illustrates the presence of multiple date formats within the dataset (e.g., DD/MM/YYYY, MM-DD-YY, YYYY/MM/DD). Such inconsistencies can lead to parsing errors, misinterpretation of time-based trends, and difficulties in data integration. Identifying and standardizing these formats is essential for accurate analysis and reporting. Figure 4.2.3 Incorrect text format

```sql
-- Remove Leading & Trailing Spaces
UPDATE credit_card
SET Exp_Type = TRIM(Exp_Type);

UPDATE credit_card
SET Exp_Type = LOWER(Exp_Type);
```

This figure highlights textual inconsistencies within the dataset, such as improper capitalization, mixed use of upper/lower case, irregular spacing, or unexpected symbols. These formatting errors can reduce data quality, hinder text analysis, and cause mismatches during processing. Detecting and correcting them ensures consistency and improves the reliability of downstream tasks.

Figure 4.2.4: Duplicate rows

```sql
-- Remove Duplicate Records
-- Identify duplicates
SELECT Client_Num, COUNT(*)
FROM credit_card
GROUP BY Client_Num
HAVING COUNT(*) > 1;

-- Delete duplicates
DELETE cc1
FROM credit_card cc1
JOIN credit_card cc2
ON cc1.Client_Num = cc2.Client_Num
AND cc1.Week_Start_Date < cc2.Week_Start_Date;
```

This figure identifies records in the dataset that are repeated across all or most columns. Duplicate rows can inflate counts, distort statistical measures, and lead to biased results if not handled properly. Detecting and removing them ensures data integrity and prevents redundancy in analysis.

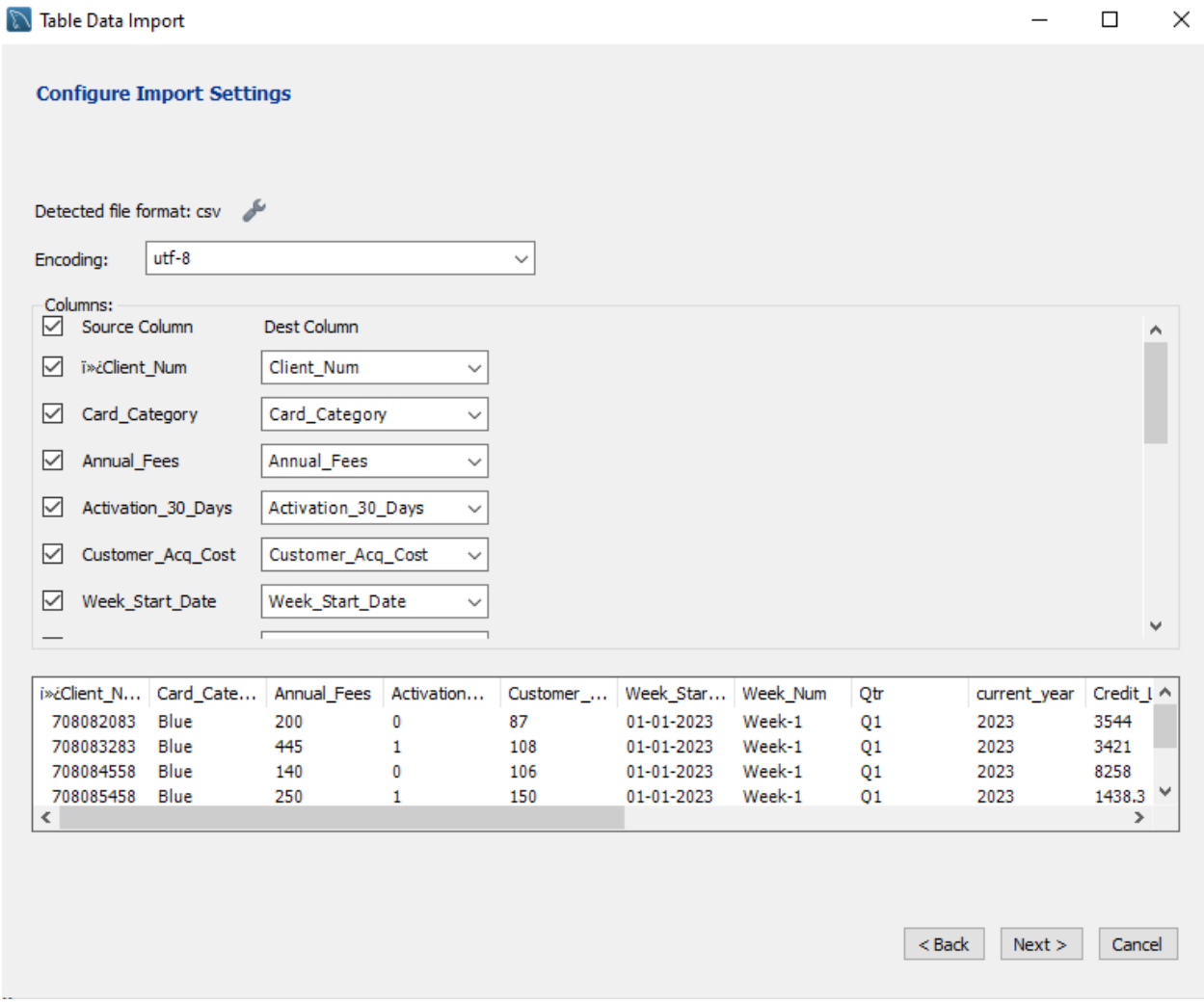
Understanding these problems was important because they directly affect the accuracy of dashboards and insights.

4.3 Data Import into SQL

The first step was to import the entire dataset into MySQL Workbench.

During the import process:

Figure 4.3.1: Data Import Summary



Correct data types were assigned to each column

Date fields were imported using mm/dd/yyyy format

Numeric fields such as Annual Fees, Total Transactions, and Credit Limit were converted to integers or decimals.

Provides an overview of the dataset imported into SQL, including row counts, column names, and detected anomalies.

SQL tables were then created for:

Figure 4.3.2: credit_card table

```sql
-- Create credit_card_detail table
CREATE TABLE credit_card (
    Client_Num INT,
    Card_Category VARCHAR(20),
    Annual_Fees INT,
    Activation_30_Days INT,
    Customer_Acq_Cost INT,
    Week_Start_Date DATE,
    Week_Num VARCHAR(20),
    Qtr VARCHAR(10),
    current_year INT,
    Credit_Limit DECIMAL(10,2),
    Total_Revolving_Bal INT,
    Total_Trans_Amt INT,
    Total_Trans_Ct INT,
    Avg_Utilization_Ratio DECIMAL(10,3),
    Use_Chip VARCHAR(10),
    Exp_Type VARCHAR(50),
    Interest_Earned DECIMAL(10,3),
    Delinquent_Acc VARCHAR(5)
);
```

The credit_card table stores customer details, card features, and transaction activity for financial analysis.

It tracks revenue drivers like fees, interest, and utilization, along with risk indicators such as delinquency.

Figure 4.3.3: customer table

```sql
-- Create customer_detail table
CREATE TABLE customer (
    Client_Num INT,
    Customer_Age INT,
    Gender VARCHAR(5),
    Dependent_Count INT,
    Education_Level VARCHAR(50),
    Marital_Status VARCHAR(20),
    State_cd VARCHAR(50),
    Zipcode VARCHAR(20),
    Car_Owner VARCHAR(5),
    House_Owner VARCHAR(5),
    Personal_Loan VARCHAR(5),
    Contact VARCHAR(50),
    Customer_Job VARCHAR(50),
    Income INT,
    Cust_Satisfaction_Score INT
);
```

These tables acted as the structured base for all further processing.

4.4 Data Cleaning Using SQL

SQL was used to perform the initial cleaning because it is fast and efficient for large datasets.

The cleaning steps included:

Figure 4.4.1: Handling Missing Values

```sql
-- Check for NULL Values
SELECT *
FROM credit_card
WHERE Client_Num IS NULL
   OR Card_Category IS NULL
   OR Total_Trans_Amt IS NULL
   OR Interest_Earned IS NULL;

-- Replace NULL Values
UPDATE credit_card
SET Interest_Earned = 0
WHERE Interest_Earned IS NULL;
```

Replacing null values with zero or meaningful defaults

Shows the count of missing entries across dataset columns, highlighting variables with significant data gaps

Using COALESCE() for missing numeric fields

Figure 4.4.2: Fixing Date Format

```sql
-- Fix Date Format
UPDATE credit_card
SET Week_Start_Date = STR_TO_DATE(Week_Start_Date, '%m/%d/%Y');
```

All date fields were converted into proper MySQL date format using:

STR_TO_DATE(week_start_date, '%m/%d/%Y')

Figure 4.4.3: Removing Duplicates

```sql
-- Remove Duplicate Records
-- Identify duplicates
SELECT Client_Num, COUNT(*)
FROM credit_card
GROUP BY Client_Num
HAVING COUNT(*) > 1;

-- Delete duplicates
DELETE cc1
FROM credit_card cc1
JOIN credit_card cc2
ON cc1.Client_Num = cc2.Client_Num
AND cc1.Week_Start_Date < cc2.Week_Start_Date;
```

Duplicates were identified and removed using:

ROW_NUMBER () OVER (PARTITION BY)

Identifies repeated records in the dataset, which can distort analysis and must be removed for accuracy.

Figure 4.4.4: Standardizing Text Values

```sql
UPDATE credit_card
SET Use_Chip = 'Chip Used'
WHERE LOWER(TRIM(Use_Chip)) IN (
    'use chip',
    'use_chip',
    'usechip',
    'chip',
    'chip used'
);
```

For example, converting “Use chip” and “use Chip” into a single format like “Chip Used”.

Highlights textual inconsistencies such as irregular capitalization, spacing, or symbols that reduce data quality.

Figure 4.4.5: Creating New Calculated Columns

Examples include:

- Total Revenue

```
Revenue = 'credit_card'[Annual_Fees] + 'credit_card'[Total_Trans_Amt] + 'credit_card'[Interest_Earned]
```

- Customer Age Group

```
AgeGroup = SWITCH(
    TRUE(),
    'customer'[Customer_Age] < 30, "20-30",
    'customer'[Customer_Age] >= 30 && 'customer'[Customer_Age] < 40, "30-40",
    'customer'[Customer_Age] >= 40 && 'customer'[Customer_Age] < 50, "40-50",
    'customer'[Customer_Age] >= 50 && 'customer'[Customer_Age] < 60, "50-60",
    'customer'[Customer_Age] >= 60, "60+",
    "unknown"
    )
```

- Expense Category

```
Total Expenses = SUM(Expenses[Amount])
```

These calculations helped in building more meaningful visualizations later.

4.5 Data Preparation in Python

After SQL-level cleaning, the data was exported into Python for deeper processing.

Figure 4.5.1: Outlier Detection and Treatment

```
import pandas as pd
import matplotlib.pyplot as plt
import sqlalchemy as sa
```

Python libraries like Pandas and NumPy were used to identify extremely high or low values in.

Figure 4.5.2: CONNECT TO SQL DATABASE

```
db = sa.create_engine("mysql+pymysql://root:Kajal%403105@localhost/credit_card_db")
```

```
credit = pd.read_sql("SELECT * FROM credit_card", db)
customer = pd.read_sql("SELECT * FROM customer", db)

print(credit.head())
print(customer.head())
```

Provides an overview of the dataset imported into SQL, including row counts, column names, and anomalies detected during ingestion

Figure 4.5.3: Total Transaction Amount

```
#3.Total Transactions
Total_transactions = credit['Total_Trans_Ct'].sum()
```

Figure 4.5.4: Accurate KPI calculation

Summarizes key performance indicators such as revenue growth, profit margin, and expense ratios, offering a snapshot of business performance

```
# 1.Total Revenue
Total_revenue = round((credit['Annual_Fees'].sum() +
                                credit['Total_Trans_Amt'].sum() +
                                credit['Interest_Earned'].sum()) / 1_000_000, 2)

#2.Total Customers
Total_customers = customer['Client_Num'].nunique()

#3.Total Transactions
Total_transactions = credit['Total_Trans_Ct'].sum()

#4.Total Interest Earned
Total_interest_million = round(credit['Interest_Earned'].sum() / 1_000_000, 2)

# Print KPIs
print(f"Total Revenue: {Total_revenue} M")
print(f"Total Customers: {Total_customers}")
print(f"Total Transactions: {Total_transactions}")
print(f"Total Interest Earned: {Total_interest_million} M")
```

```
Total Revenue: 55.32 M
Total Customers: 10108
Total Transactions: 655651
Total Interest Earned: 7.84 M
```

Outliers were either corrected or capped using the IQR method.

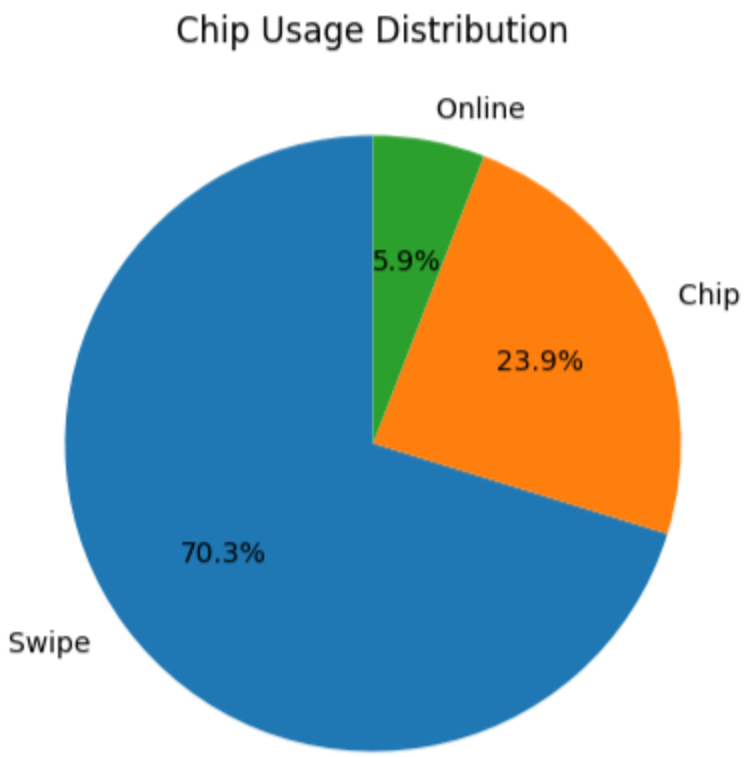
Checking Data Distribution

Histograms and boxplots were used to study transaction patterns and spending behavior.

Figure 4.5.6: Transaction Mode Distribution

2.Chip vs Non-Chip Usage

```
chip_usage = df["Use_Chip"].value_counts()

plt.figure(figsize=(5, 5))
chip_usage.plot(kind='pie', autopct='%1.1f%%', startangle=90)
plt.title("Chip Usage Distribution")
plt.ylabel("")
plt.show()
```

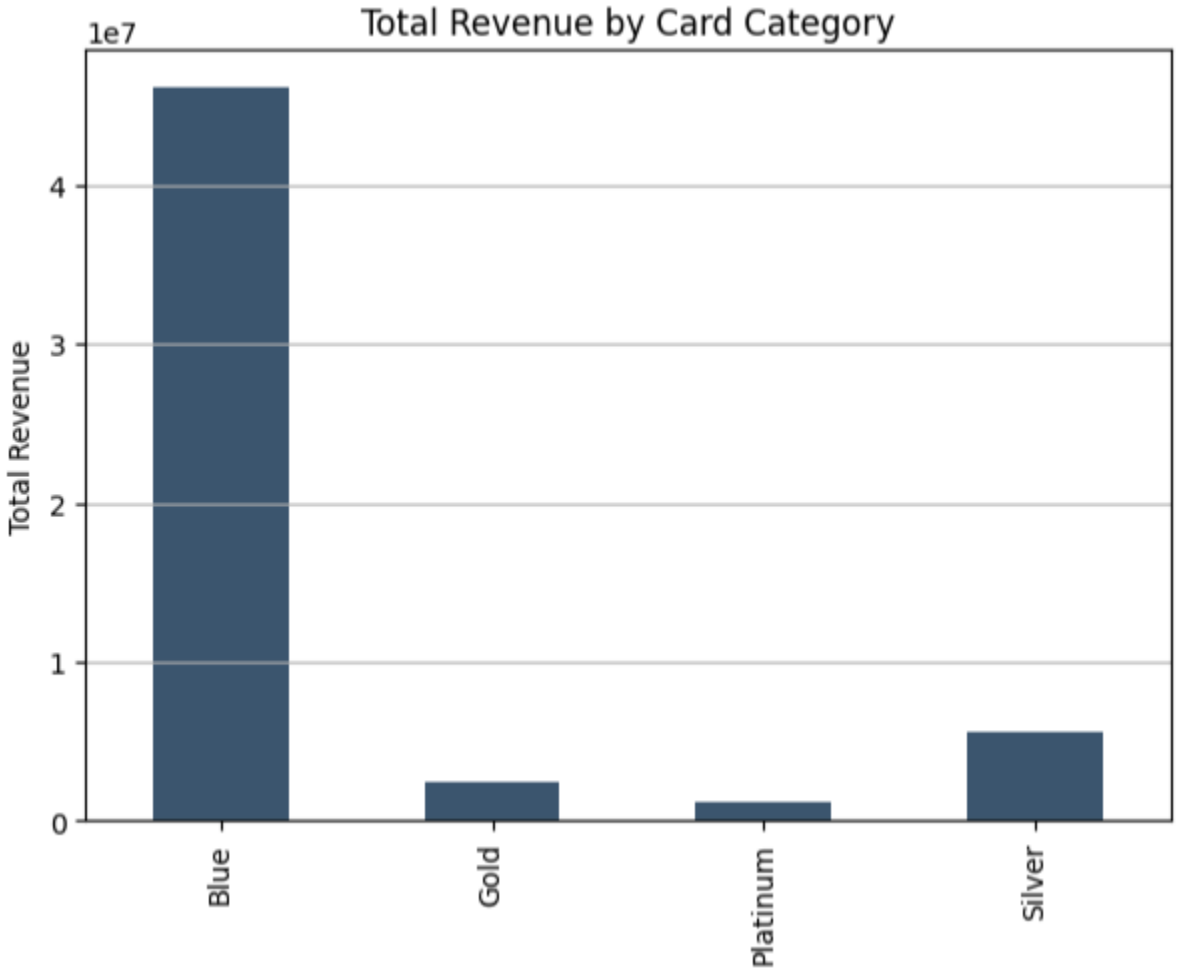


This figure compares the usage of different payment modes—**Swipe, Chip, and Online**. It highlights that **Swipe transactions are the most frequently used**, indicating customer preference for card-swiping over chip insertion or online payments. The visualization helps identify dominant transaction behavior, which can guide decisions on infrastructure, customer experience improvements, and fraud monitoring.

Figure 4.5.7: Encoding Categorical Fields

1.Revenue by Card Category ¶

```
Revenue_by_card = credit.groupby("Card_Category")["Revenue"].sum()

plt.figure(figsize=(7, 5))
Revenue_by_card.plot(kind='bar', color='#3B556E')
plt.title("Total Revenue by Card Category")
plt.xlabel("Card Category")
plt.ylabel("Total Revenue")
plt.grid(axis='y')
plt.show()
```



Text fields such as education level, marital status, and card type were encoded to make them analysis-ready.

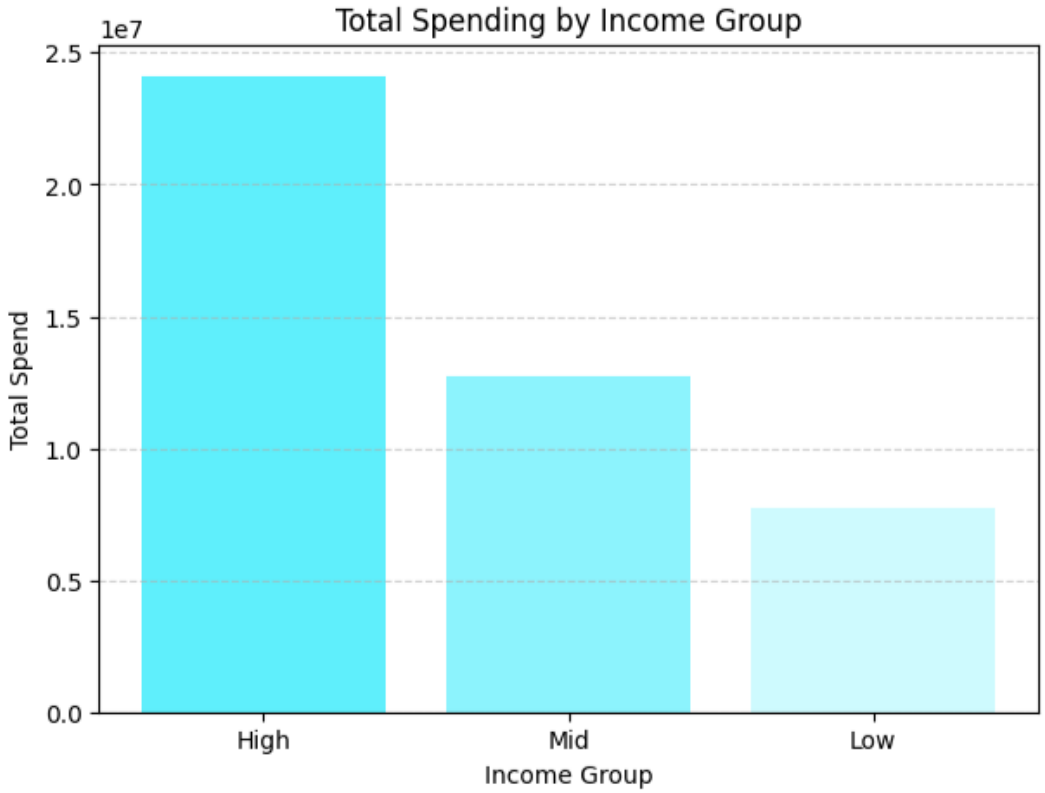
This figure compares revenue contributions across different card categories—**Blue, Gold, Platinum, and Silver**. It highlights that **Blue cards are the most frequently used by customers**, generating the highest share of revenue among all categories. The visualization provides insights into customer preferences, showing that Blue cards dominate transaction activity, while Gold, Platinum, and Silver contribute smaller portions. This helps in understanding card performance and guiding strategies for promotions or targeted customer engagement.

Figure 4.5.8: Total Spending by Income Group

```
# 1. Create Income_Group column
customer['Income_Group'] = pd.cut(
    customer['Income'],
    bins=[0, 35000, 70000, float('inf')],
    labels=['Low', 'Mid', 'High'],
    right=False
)

# 2. Merge customer and credit_card tables
merged = customer.merge(credit, on='Client_Num', how='inner')

# 3. Group by Income_Group and sum Total_Trans_Amt
income_group_spending = (
    merged.groupby('Income_Group',observed=False)['Total_Trans_Amt']
    .sum()
    .reset_index(name='Total_Spend')
    .sort_values(by='Total_Spend', ascending=False)
)
```

```
print(income_group_spending)

# Bar chart
plt.figure(figsize=(7, 5))
plt.bar(income_group_spending['Income_Group'],
        income_group_spending['Total_Spend'],
        color=['#5FEFFC', '#8CF3FD', '#CEFAFE'])

plt.title('Total Spending by Income Group')
plt.xlabel('Income Group')
plt.ylabel('Total Spend')
plt.grid(axis='y', linestyle='--', alpha=0.6)

plt.show()
```

```
  Income_Group  Total_Spend
2         High     24099963
1          Mid     12706750
0          Low      7715300
```

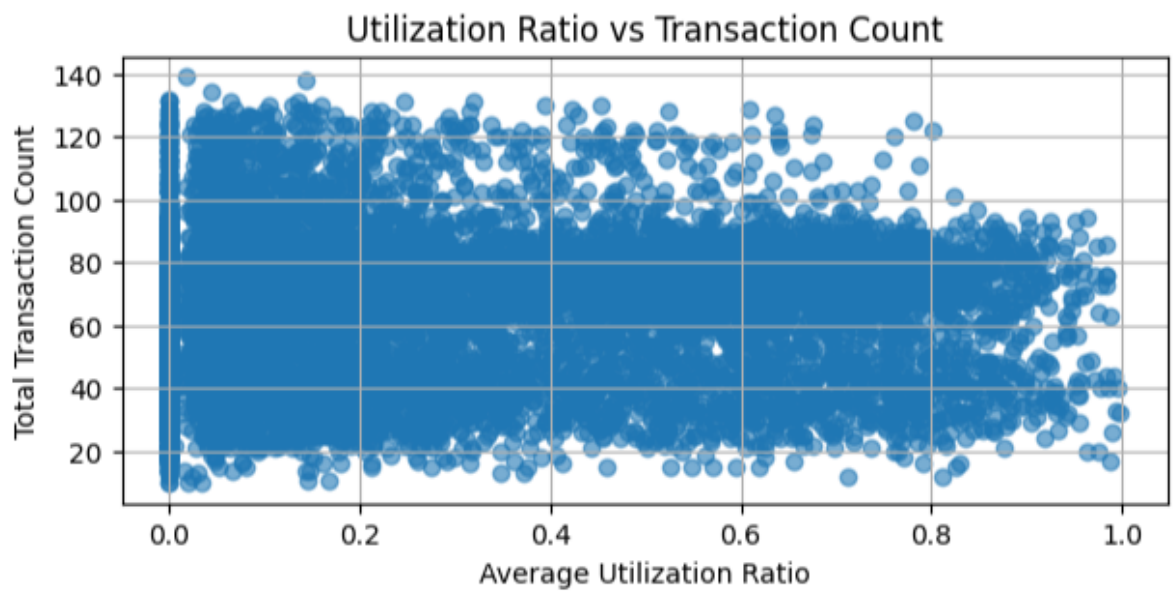


A bar chart comparing total spending across income groups, with the tallest bar representing the group that spends the most.

This figure 4.5.8 illustrates the distribution of total transaction spending across three income groups—**Low, Mid, and High**. By categorizing customers based on income ranges, the chart highlights which group contributes the most to overall spending. The visualization makes it easy to compare spending behavior across segments, showing that higher-income groups typically drive greater transaction amounts, while lower-income groups contribute less. This insight supports customer segmentation, targeted marketing, and financial analysis.

Figure 4.5.9: Utilization Ratio vs Transaction Count

```
plt.figure(figsize=(7, 3))
plt.scatter(df["Avg_Utilization_Ratio"], df["Total_Trans_Ct"], alpha=0.6)
plt.title("Utilization Ratio vs Transaction Count")
plt.xlabel("Average Utilization Ratio")
plt.ylabel("Total Transaction Count")
plt.grid(True)
plt.show()
```



A scatter plot illustrating how transaction count varies with utilization ratio, highlighting customer usage trends.

This figure presents a scatter plot showing the relationship between customers' **average utilization ratio** (credit usage relative to available limit) and their **total transaction count**. Each point represents a customer, with the horizontal axis indicating utilization ratio and the vertical axis showing transaction volume. The visualization helps identify patterns such as whether higher utilization ratios are associated with more frequent transactions, clusters of typical customer behavior, and potential outliers.

4.6 Exporting Clean Data to Power BI

Once SQL and Python cleaning was completed, the final dataset was loaded into Power BI.

This cleaned dataset allowed:

- Smooth visualization
- Accurate KPI calculation
- Faster dashboard performance
- Better trend and revenue analysis

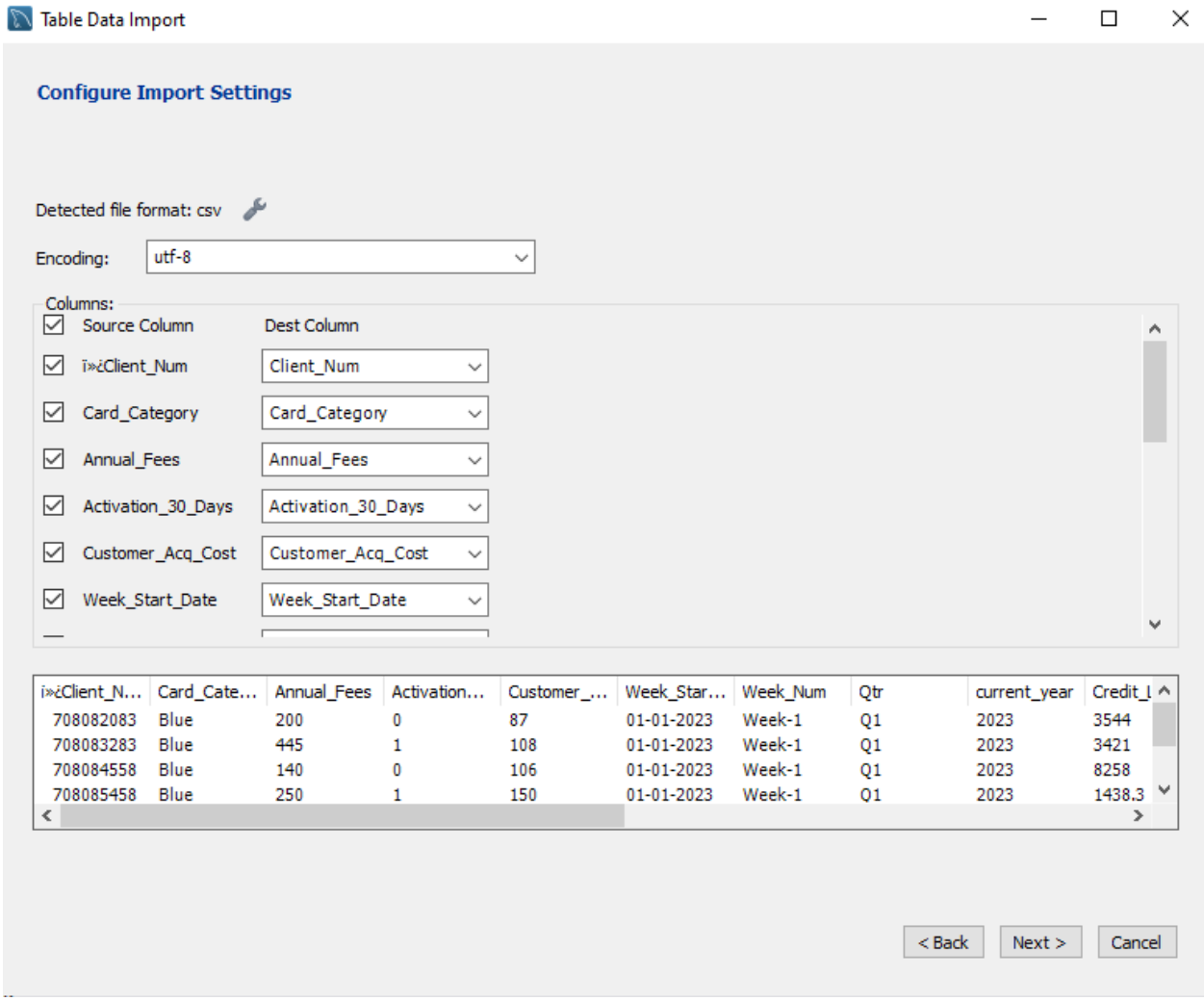
Power BI was then connected directly to the SQL server to allow automatic refreshes in the future.

Summary of Data Preparation Workflow

The entire process can be summarized in four simple steps:

Import raw data → into SQL

Figure 4.6.1: SQL Raw Data Import Summary



Shows the process of loading raw data files into SQL tables, enabling structured storage and analysis.

Clean and structure data → using SQL queries

Figure 4.6.2: Data Cleaning and Structuring in SQL

```sql
-- Identify Missing Values
SELECT *
FROM credit_card
WHERE Client_Num IS NULL
   OR Total_Trans_Amt IS NULL
   OR Credit_Limit IS NULL
   OR Interest_Earned IS NULL;

-- Handle Missing Values
UPDATE credit_card
SET Interest_Earned = 0
WHERE Interest_Earned IS NULL;

-- Remove Duplicate Records
SELECT Client_Num, COUNT(*)
FROM credit_card
GROUP BY Client_Num
HAVING COUNT(*) > 1;
```

```sql
DELETE cc1
FROM credit_card cc1
JOIN credit_card cc2
ON cc1.Client_Num = cc2.Client_Num
AND cc1.Week_Start_Date < cc2.Week_Start_Date;

-- Fix Incorrect Date Format
UPDATE credit_card
SET Week_Start_Date = STR_TO_DATE(Week_Start_Date, '%m/%d/%Y');

-- Clean Incorrect Text Formats
UPDATE credit_card
SET Use_Chip = 'Chip Used'
WHERE LOWER(TRIM(Use_Chip)) IN ('use chip', 'use_chip', 'chip', 'chip used');
```

This query demonstrates how raw data is transformed into a clean and structured format using SQL queries. It highlights operations such as removing duplicates, handling missing values, standardizing text and date formats, and creating derived columns. The process ensures that the dataset is consistent, reliable, and ready for analysis or reporting.

Enhance and refine data → using Python

Figure 4.6.3: Data Enhancement and Refinement in Python

```
credit['Revenue'] = credit['Annual_Fees'] + credit['Total_Trans_Amt'] + credit['Interest_Earned']
```

```
print(credit.columns)
```

```
Index(['Client_Num', 'Card_Category', 'Annual_Fees', 'Activation_30_Days',
       'Customer_Acq_Cost', 'Week_Start_Date', 'Week_Num', 'Qtr',
       'current_year', 'Credit_Limit', 'Total_Revolving_Bal',
       'Total_Trans_Amt', 'Total_Trans_Ct', 'Avg_Utilization_Ratio',
       'Use_Chip', 'Exp_Type', 'Interest_Earned', 'Delinquent_Acc', 'Revenue'],
      dtype='object')
```

3.Merge tables

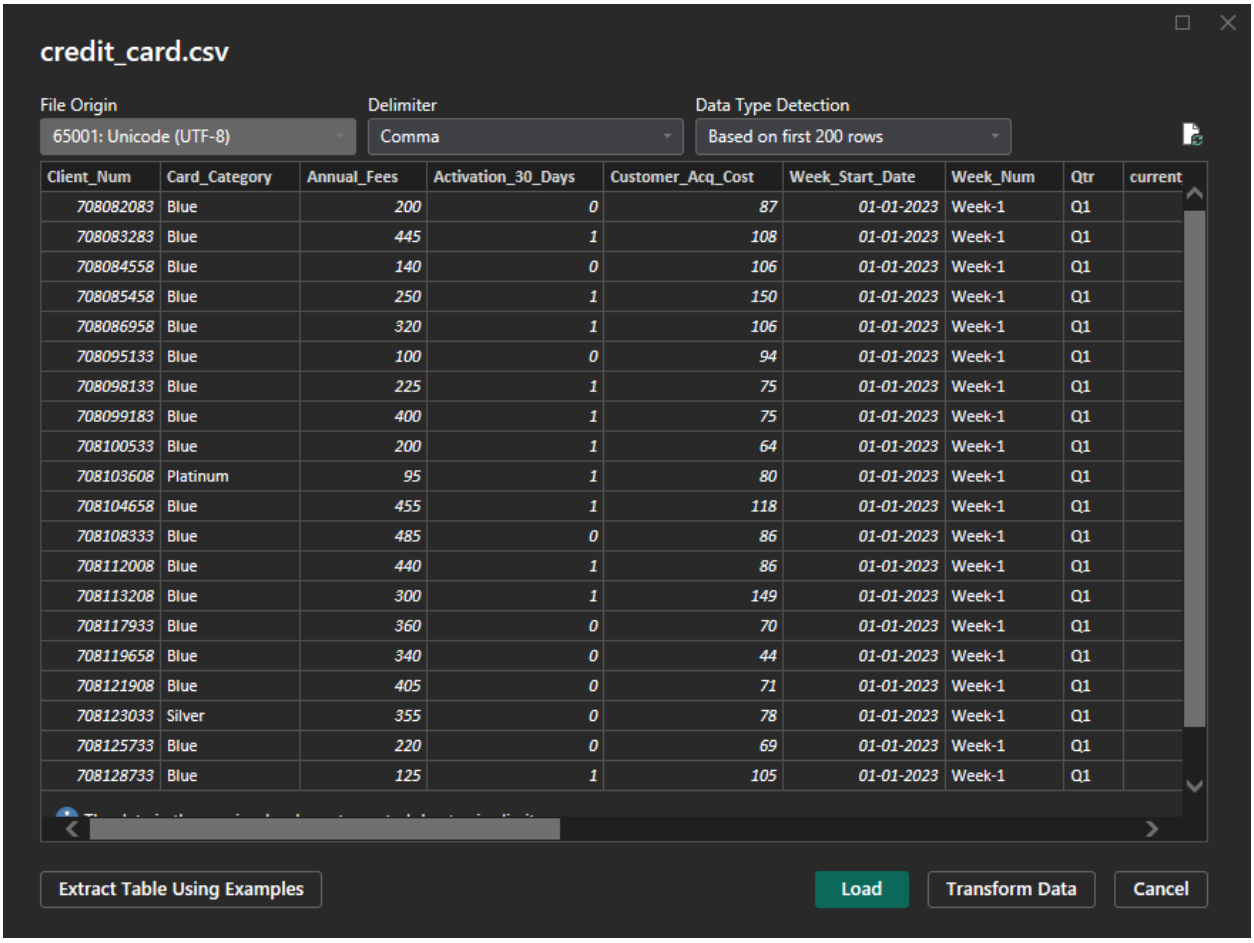
```
df = credit.merge(customer, on="Client_Num", how="inner")

print("Data Loaded Successfully!")
print(df.head())
```

```
Data Loaded Successfully!
   Client_Num Card_Category  Annual_Fees  Activation_30_Days  \
0   708082083          Blue          200                   0
1   708083283          Blue          445                   1
2   708084558          Blue          140                   0
3   708085458          Blue          250                   1
4   708086958          Blue          320                   1

   Customer_Acq_Cost Week_Start_Date Week_Num Qtr  current_year  Credit_Limit  \
0                 87      2023-01-01   Week-1  Q1          2023        3544.0
```

```python
# 1. Create Income_Group column
customer['Income_Group'] = pd.cut(
    customer['Income'],
    bins=[0, 35000, 70000, float('inf')],
    labels=['Low', 'Mid', 'High'],
    right=False
)

# 2. Merge customer and credit_card tables
merged = customer.merge(credit, on='Client_Num', how='inner')

# 3. Group by Income_Group and sum Total_Trans_Amt
income_group_spending = (
    merged.groupby('Income_Group',observed=False)['Total_Trans_Amt']
    .sum()
    .reset_index(name='Total_Spend')
    .sort_values(by='Total_Spend', ascending=False)
)

print(income_group_spending)
```

This figure demonstrates how Python is used to improve data quality and usability. It highlights operations such as feature engineering, normalization, handling missing values, and creating new derived variables. By refining the dataset with Python libraries like **pandas**, and **numpy** the process ensures cleaner, more structured data that is ready for advanced analysis and modeling.

Figure 4.6.4: Power BI Data Loading and Visualization



Client_Num	Card_Category	Annual_Fees	Activation_30_Days	Customer_Acq_Cost	Week_Start_Date	Week_Num	Qtr	current
708082083	Blue	200	0	87	01-01-2023	Week-1	Q1	
708083283	Blue	445	1	108	01-01-2023	Week-1	Q1	
708084558	Blue	140	0	106	01-01-2023	Week-1	Q1	
708085458	Blue	250	1	150	01-01-2023	Week-1	Q1	
708086958	Blue	320	1	106	01-01-2023	Week-1	Q1	
708095133	Blue	100	0	94	01-01-2023	Week-1	Q1	
708098133	Blue	225	1	75	01-01-2023	Week-1	Q1	
708099183	Blue	400	1	75	01-01-2023	Week-1	Q1	
708100533	Blue	200	1	64	01-01-2023	Week-1	Q1	
708103608	Platinum	95	1	80	01-01-2023	Week-1	Q1	
708104658	Blue	455	1	118	01-01-2023	Week-1	Q1	
708108333	Blue	485	0	86	01-01-2023	Week-1	Q1	
708112008	Blue	440	1	86	01-01-2023	Week-1	Q1	
708113208	Blue	300	1	149	01-01-2023	Week-1	Q1	
708117933	Blue	360	0	70	01-01-2023	Week-1	Q1	
708119658	Blue	340	0	44	01-01-2023	Week-1	Q1	
708121908	Blue	405	0	71	01-01-2023	Week-1	Q1	
708123033	Silver	355	0	78	01-01-2023	Week-1	Q1	
708125733	Blue	220	0	69	01-01-2023	Week-1	Q1	
708128733	Blue	125	1	105	01-01-2023	Week-1	Q1	

Load clean data → into Power BI for visualization

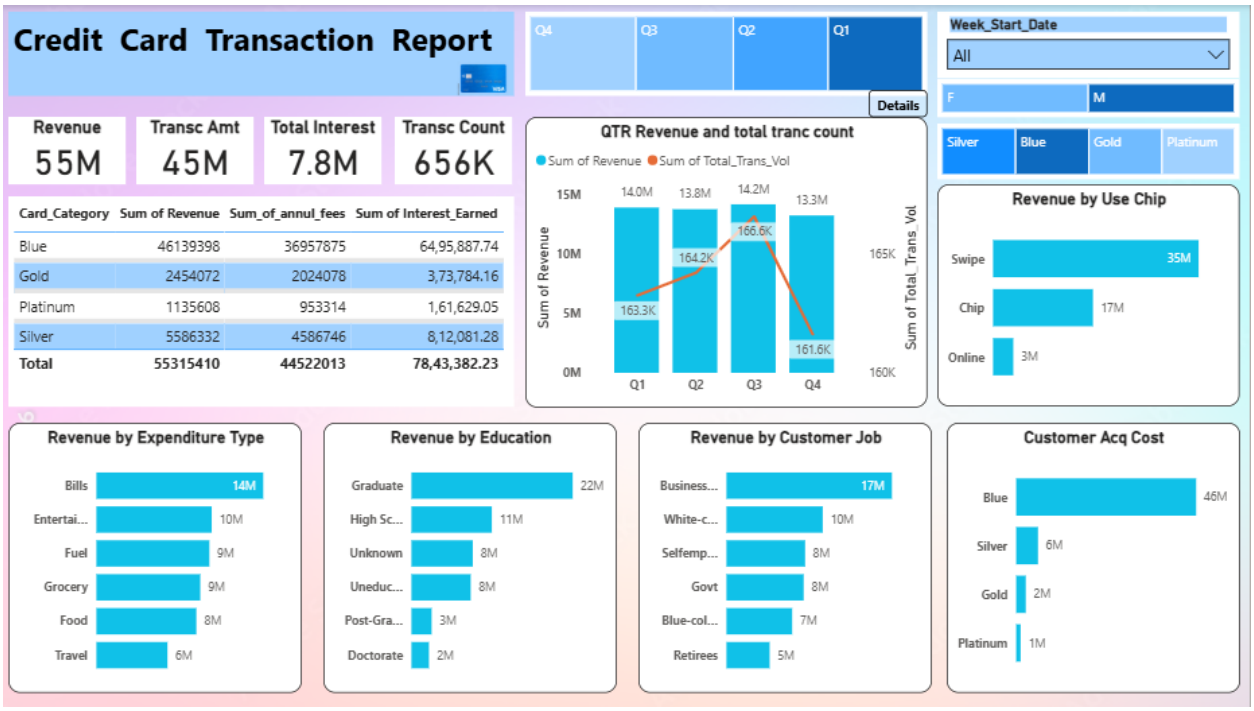
This figure shows how cleaned and structured data is imported into Power BI for analysis. It highlights the process of connecting to the prepared dataset, loading tables, and building interactive dashboards. By leveraging Power BI's visualization tools, the clean data is transformed into charts, graphs, and KPIs that provide actionable insights and support decision-making.

4.7 DASHBOARD DESIGN & DATA INSIGHTS

This chapter explains how the Power BI dashboard was designed and what kind of insights were obtained from it. The dashboard is the most important part of the project because it converts complex data into an easy-to-understand visual format. Instead of reading tables or SQL outputs, users can simply view charts and KPIs to understand the overall performance of the credit card business.

The Power BI dashboard transforms clean data into interactive visuals, delivering insights on revenue, transactions, customer segments, and quarterly trends for informed decision-making.

Figure 4.7.1: Credit Card Transaction Report Dashboard



This dashboard provides a comprehensive overview of credit card transaction performance using interactive Power BI visuals. It begins with four key KPIs—**Revenue, Transaction Amount,**

Total Interest, and Transaction Count—offering a quick snapshot of overall financial health. Supporting bar charts break down revenue by **expense category, card category, customer job role, and education level**, as well as **customer acquisition cost**, enabling detailed segmentation and comparison across customer groups. A **treemap visualization by quarters** highlights seasonal trends and revenue distribution over time. Together, these elements deliver actionable insights into customer behavior, spending patterns, and profitability, helping stakeholders identify high-value segments and optimize business strategies.

Figure 4.7.2: Credit Card Customer Report Dashboard

This dashboard provides a detailed view of customer demographics and behavior related to credit card usage. It includes visualizations that break down customers by **income group, education level, occupation, and age segment**, offering insights into spending patterns and preferences across different profiles. Key metrics such as **transaction count, average utilization ratio, and**

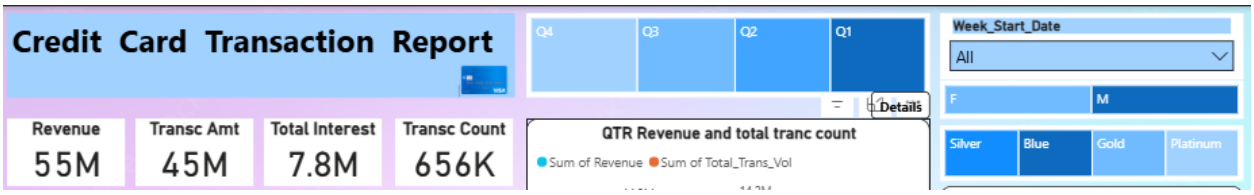
total spend are highlighted to show how customer characteristics influence credit card activity. The dashboard enables stakeholders to identify high-value customer segments, monitor usage trends, and design targeted strategies for acquisition, retention, and engagement.

4.8 Dashboard Design Approach

The main goal while designing the dashboard was clarity and simplicity. The dashboard was created in such a way that even a non-technical user can understand it easily. A clean layout was selected with proper spacing, readable colors, and meaningful titles.

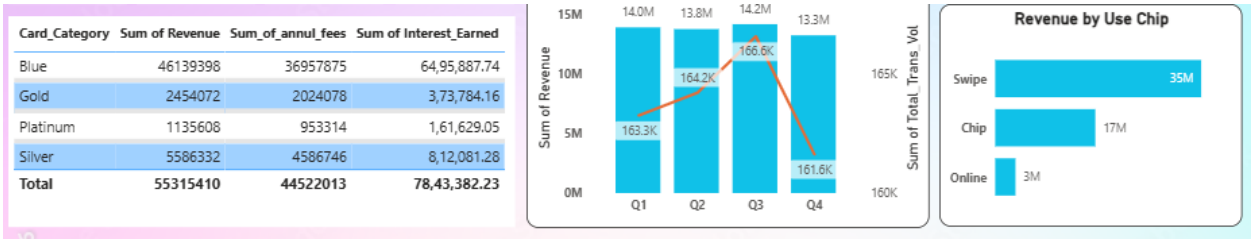
The dashboard is divided into multiple sections:

Figure 4.8.1: Top section



Displays key performance indicators (KPIs)

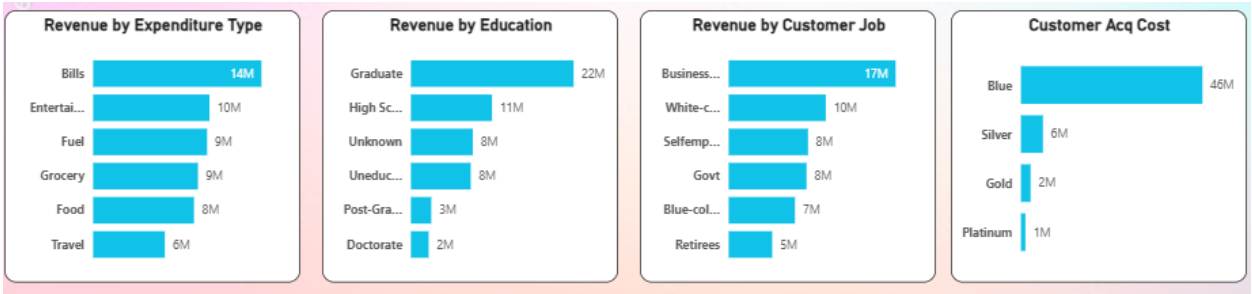
Figure 4.8.2: Middle section



Card_Category	Sum of Revenue	Sum_of_annul_fees	Sum of Interest_Earned
Blue	46139398	36957875	64,95,887.74
Gold	2454072	2024078	3,73,784.16
Platinum	1135608	953314	1,61,629.05
Silver	5586332	4586746	8,12,081.28
Total	**55315410**	**44522013**	**78,43,382.23**

Shows trend and category-based analysis

Figure 4.8.3: Bottom section



Displays detailed customer and transaction insights.

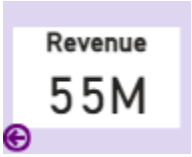
Filters and slicers were added so that users can interact with the data. Users can filter the dashboard by card category, gender, education level, expense type, and year.

4.9 Key Performance Indicators (KPIs)

The following KPIs were included in the dashboard:

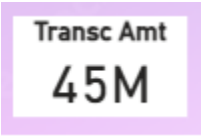
Figure 4.9.1: Total Revenue

```
Revenue = 'credit_card'[Annual_Fees] + 'credit_card'[Total_Trans_Amt] + 'credit_card'[Interest_Earned]
```



Represents the overall revenue generated from **annual fees, interest earned, and transaction amounts**, providing a complete measure of financial performance.

Figure 4.9.2: Total Transaction Amount

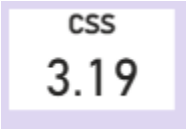


Represents the overall value of all credit card transactions processed, reflecting customer spending activity and transaction volume.

Figure 4.9.3: Total Transaction Count

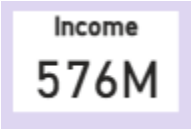
Represents the total number of credit card transactions processed, reflecting overall customer activity and usage frequency

Figure 4.9.4: Average Credit Utilization Ratio



Measures the average proportion of credit limit used by customers, indicating overall credit usage behavior and risk exposure.

Figure 4.9.5: Total Income Earned



Represents the overall income generated from all credit card activities, including **annual fees, interest earned, and transaction-related revenue**, providing a complete measure of financial performance.

Figure 4.9.6: Total Interest Earned

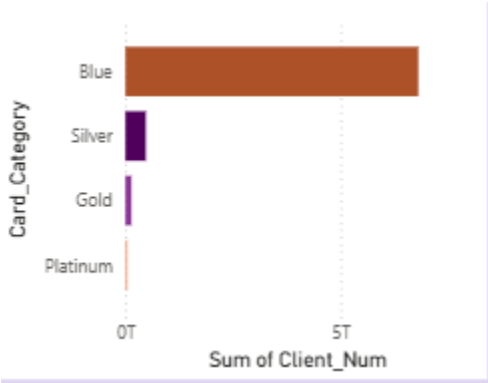
Represents the total revenue generated from interest charges on credit card balances, reflecting profitability from customer borrowing behavior.

These KPIs give a quick overview of financial performance and customer behavior.

4.10 Revenue Analysis

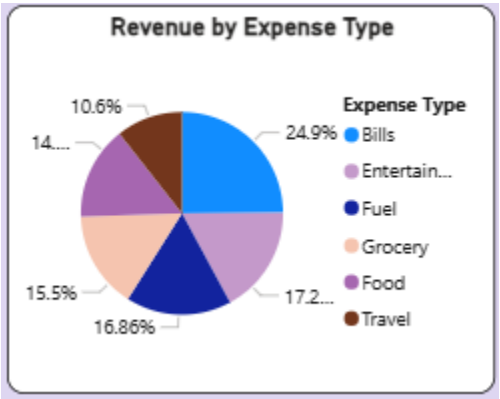
This analysis highlights the different components and drivers of credit card revenue. It combines **annual fees, interest earned, and transaction amounts** to present a complete view of total revenue. Supporting visuals break down revenue by **expense category, card category, customer demographics (job, education, income group), and acquisition cost**, along with quarterly trends. The insights help identify which segments and activities contribute most to profitability, uncover seasonal patterns, and guide strategies for optimizing customer engagement and revenue growth.

Figure 4.10.1: Card Category (Silver, Gold, Platinum, etc.)



This represents the different tiers of credit cards offered to customers, such as Silver, Gold, and Platinum. Each category reflects a level of benefits, rewards, and usage patterns. By analyzing card categories, you can see how customer spending and revenue contribution vary across tiers, helping to identify which card types are most popular and which generate the highest value for the business.

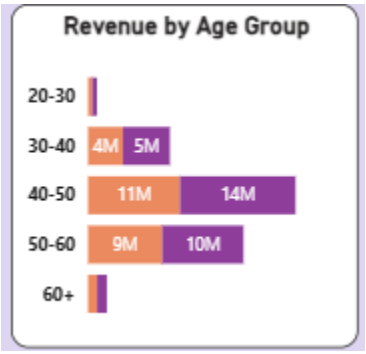
Figure 4.10.2: Expense Type (Shopping, Travel, Fuel, Bills, etc.)



This represents the different categories of spending made through credit cards, such as shopping, travel, fuel, and bill payments. By analyzing expense types, the dashboard shows where customers are using their cards the most, helping to identify spending trends, popular categories,

and areas that drive the highest revenue.

Figure 4.10.3: Customer Age Group



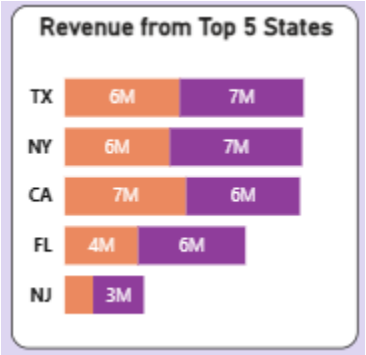
This figure represents the segmentation of customers based on their age ranges, such as 20-30, 30–40, 40–50,50-60 and 60+. By analyzing age groups, the dashboard highlights how different generations use credit cards, their spending habits, and contribution to overall revenue. It helps identify which age segments are most active, which drive higher transaction volumes, and where targeted marketing or product strategies can be applied.

Figure 4.10.4: Revenue by Income Range

This shows how credit card revenue is distributed across different customer income groups. By breaking down revenue by ranges such as low, medium, and high income, the dashboard

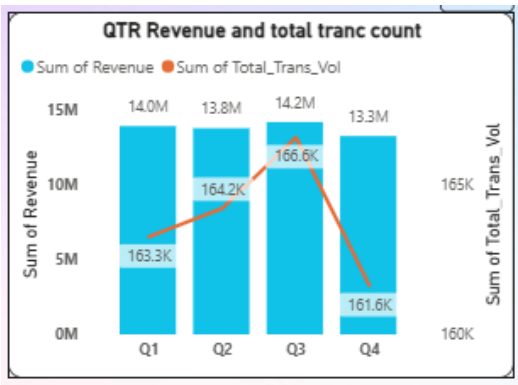
highlights which segments contribute the most to overall profitability. It helps identify spending capacity, usage behavior, and the financial value of customers in each income bracket.

Figure 4.10.5: Revenue from Top 5 states



This metric highlights the total revenue contribution from the five states with the highest credit card activity. By focusing on the top performers, the dashboard shows regional trends, customer spending behavior, and geographic areas driving the most profitability. It helps identify strong markets, compare performance across states, and guide strategies for regional targeting and resource allocation

Figure 4.10.6: QTR Revenue and Total Transaction Count



This section tracks revenue and transaction activity across different quarters of the year. By combining quarterly revenue with the total number of transactions, the dashboard highlights seasonal trends, customer spending behavior, and periods of peak performance. It helps identify which quarters drive the highest profitability and transaction volumes, offering valuable insights for planning, forecasting, and strategy alignment.

From the analysis, it can be observed that customers with higher credit limits and frequent transactions contribute more to revenue. Certain card categories generate higher annual fees and interest income.

4.11 Customer Behavior Insights

Customer behavior analysis is one of the most valuable parts of the dashboard.

- **Preferred Card Category:** The **Blue Card** is the most widely used, showing strong customer preference for this tier. It suggests that customers find the benefits and features of this card most suitable for their needs.
- **Expense Trends:** The highest revenue comes from **bills, entertainment, and fuel**, indicating that customers rely heavily on credit cards for essential payments, lifestyle spending, and daily commuting needs.
- **Age Group Activity:** Customers in the **40–50 age group** are the most active, reflecting mature professionals with stable incomes who use credit cards frequently for both necessities and leisure.
- **Income Range Contribution:** Revenue is predominantly driven by **high-income customers**, showing that affluent segments contribute the largest share of profitability through higher transaction amounts and card usage.

- **Geographic Insights:** The **top five states—Texas, New York, California, Florida, and New Jersey—** generate the most revenue. This highlights regional hubs of strong customer engagement and spending power.
- **Seasonal Behavior:** Both **quarterly revenue and transaction counts increase in Q3**, pointing to seasonal peaks in spending, possibly linked to holidays, travel, or back-to-school expenses.
- **Preferred Payment Mode:** Customers mostly use **chip swipe** when making transactions. This shows a strong reliance on traditional card-present methods compared to tap-to-pay or online transactions.

The dashboard helps identify loyal customers as well as customers who may require attention.

4.12 Credit Utilization & Risk Analysis

Credit utilization ratio is a key risk indicator.

From the dashboard:

Customers with high utilization ratios are more likely to become delinquent

Delinquent accounts show irregular transaction behavior

Monitoring utilization helps reduce financial risk

These insights help banks take early action to prevent losses.

4.13 Use of Filters and Interactivity

The dashboard allows users to:

- Drill down into specific customer segments

- Compare performance across card categories
- Analyze trends over time
- Quickly identify high-risk and high-value customers

This interactivity makes the dashboard a powerful decision-support tool.

4.14 Summary of Dashboard Insights

The dashboard provides a complete view of:

1. Overall Revenue Performance

The dashboard shows that **total revenue is mainly driven by transaction amounts**, followed by interest earned and annual fees. This indicates that **customer spending behavior** plays a bigger role in revenue generation than fixed charges like annual fees.

2. Transaction Behavior

Customers with **higher transaction counts** also show **higher total transaction amounts**, indicating consistent card usage. A small group of customers contributes a disproportionately large share of transactions, highlighting the presence of **high-value customers**.

3. Card Category Analysis

Premium card categories generate **higher revenue per customer**, while standard cards contribute through **higher customer volume**. This suggests that premium cards are profitable on an individual level, whereas basic cards help maintain a stable revenue base.

4. Customer Satisfaction Impact

Higher customer satisfaction scores are associated with **increased transaction frequency and lower delinquency rates**. This highlights the importance of customer experience in improving financial performance.

5. Business Recommendations

Based on the dashboard insights:

- Focus marketing efforts on **high-value and premium card users**
- Monitor customers with **high utilization ratios** to reduce delinquency risk
- Promote **chip-based transactions** for security and higher transaction value
- Introduce loyalty programs to improve **customer satisfaction and retention**

"The dashboard reveals that customer spending behavior, credit utilization, and delinquency status are the key drivers of revenue and risk in credit card operations."

It helps banks move from raw data to clear, actionable insights.

CHAPTER 5: FINDINGS AND CONCLUSION

This chapter discusses the results obtained from the Credit Card Financial Analysis project and explains their significance. The results are based on the analysis performed using SQL, Python, and the Power BI dashboard. By combining transaction data with customer demographic information, meaningful patterns related to revenue, customer behavior, and financial risk were identified.

5.1 Findings:

Overall Financial Performance

The analysis shows that credit card revenue mainly comes from three sources: annual fees, interest earned, and transaction volume. Customers who frequently use their cards and maintain higher credit utilization ratios contribute significantly to total revenue. Card categories with higher annual fees generate more income, even if the number of customers in those categories is lower.

The dashboard clearly highlights that transaction amount and transaction count have a strong impact on revenue. Customers who perform more transactions tend to generate higher interest and fee-based income for the bank.

Advanced Analytics for Credit Card Operations

Customer Spending Behavior

One of the key findings of this project is that customer spending behavior varies significantly based on age, income, and card category. Younger customers generally make more frequent transactions, while older customers tend to have higher average transaction values.

Higher-income customers usually have higher credit limits and spend more, which results in higher transaction amounts. These customers are often associated with premium card categories and contribute more to revenue.

Credit Utilization and Risk Analysis

Credit utilization ratio plays an important role in identifying financial risk. Customers with high utilization ratios are more likely to miss payments or become delinquent. The analysis shows a clear relationship between high utilization and delinquent accounts.

Delinquent customers often display irregular spending patterns and inconsistent payment behavior. Identifying such customers early can help banks take preventive measures, such as sending reminders or adjusting credit limits.

Chip Usage and Transaction Patterns

The analysis of chip-based transactions shows that customers who use chip-enabled payments tend to make transactions more frequently. Chip usage is associated with safer and more secure transactions, which increases customer confidence.

This insight suggests that promoting secure payment methods can encourage more frequent card usage and improve customer trust.

Expense Type and Lifestyle Insights

Expense type analysis reveals clear lifestyle patterns among customers. Spending is highest in categories such as shopping, travel, food, and fuel. These spending habits help banks understand customer preferences and design targeted offers.

For example, customers who spend more on travel can be offered travel-related rewards, while customers who spend more on fuel can be given fuel cashback offers.

Customer Satisfaction Insights

Customer satisfaction scores provide important feedback about the quality of services. The analysis indicates that satisfied customers are more likely to use their credit cards regularly and maintain healthy payment behavior.

Low satisfaction scores may indicate service issues or dissatisfaction with fees, which require attention to prevent customer churn.

Discussion of Key Findings

The results of this project highlight the importance of combining customer data with transaction data. Isolated analysis of either dataset does not provide a complete picture. By linking both tables, deeper insights into customer behavior, revenue generation, and risk management were achieved.

The use of SQL ensured accurate data processing, Python improved data reliability, and Power BI made insights easily understandable through visualizations.

5.2 Summary

Overall, customer behavior reflects a mix of **essential spending (bills, fuel)** and **lifestyle choices (entertainment, travel)**, with strong engagement from middle-aged, high-income segments. Regional concentration in a handful of states drives the bulk of revenue, while seasonal peaks in Q3 highlight predictable spending cycles.

The reliance on **chip swipe** shows slower adoption of newer payment technologies, while utilization patterns reveal both opportunities for revenue growth and areas of potential credit risk.

5.3 Conclusion

This project focused on analyzing credit card financial data to gain meaningful insights using SQL, Python, and Power BI. The main goal was to transform raw customer and transaction data into useful information that can support better decision-making for banks and financial institutions.

Throughout the project, data was collected, cleaned, processed, and visualized in a structured manner. SQL was used to store and manage the data efficiently, Python helped in cleaning and preparing the data, and Power BI was used to create interactive dashboards. This end-to-end approach ensured accuracy, clarity, and consistency in the analysis.

The analysis revealed important patterns related to customer spending behavior, revenue generation, and financial risk. Factors such as transaction amount, credit utilization ratio, and card category were found to have a strong impact on revenue. The project also highlighted the importance of monitoring delinquent accounts and high utilization customers to reduce potential financial losses.

By combining customer demographic data with transaction data, a complete picture of customer behavior was achieved. The Power BI dashboard made complex financial information easy to understand and allowed users to explore insights through interactive visuals. Overall, the project successfully demonstrated how data analytics can add value to financial decision-making.

CHAPTER 6. RECOMMENDATIONS AND LIMITATIONS OF THE STUDY

This chapter presents practical recommendations based on the findings of the credit card financial analysis and also discusses the limitations faced during the study. The recommendations are aimed at helping financial institutions improve revenue performance, reduce credit risk, and enhance customer satisfaction. The limitations highlight the constraints of the current analysis and provide clarity on areas where further improvements can be made in future studies.

6.1 Recommendations of the Study

Based on the analysis carried out using SQL, Python, and Power BI, several important insights were identified related to customer behavior, transaction patterns, credit utilization, and delinquency trends. These insights form the basis for the following recommendations.

1. Focus on High-Value Customer Segments

The analysis shows that certain customer segments contribute significantly more to revenue, particularly customers with stable income levels and moderate credit utilization ratios. Financial institutions should focus on retaining these high-value customers by offering personalized benefits such as reward points, cashback offers, and loyalty programs. Customized offers can encourage higher transaction volumes while maintaining responsible credit usage.

2. Improve Credit Risk Management

Customers with high credit utilization ratios were found to have a higher likelihood of delinquency. Banks should closely monitor such customers and implement early warning

systems to identify potential payment issues. Proactive measures such as payment reminders, flexible repayment options, or temporary credit limit adjustments can help reduce delinquency rates and financial losses.

3. Optimize Credit Limit Allocation

The study indicates that not all customers fully utilize their assigned credit limits, while some customers frequently approach their maximum limit. Financial institutions can optimize credit limit allocation by periodically reviewing customer spending behavior, income level, and repayment history. Adjusting credit limits based on actual usage can improve customer satisfaction while minimizing risk exposure.

4. Encourage Secure Transaction Methods

Chip-based transactions contributed a significant portion of total transaction value, indicating customer preference for secure payment methods. Banks should continue to promote chip-based and contactless payment technologies by educating customers on their benefits and ensuring smooth transaction experiences. This can improve transaction volume and enhance trust in digital payment systems.

5. Enhance Customer Experience and Satisfaction

The analysis highlights a positive relationship between customer satisfaction and transaction frequency. Customers with higher satisfaction scores tend to be more active and show lower delinquency rates. Financial institutions should invest in improving customer service, resolving issues quickly, and offering user-friendly digital platforms. A better customer experience can lead to higher engagement and long-term profitability.

6. Use Data-Driven Decision Making

The project demonstrates the value of using data analytics tools for financial decision-making. Banks should adopt similar analytical approaches in their regular operations to monitor performance, identify emerging trends, and respond quickly to changes in customer behavior. Dashboards created using Power BI can be used by management for real-time monitoring and strategic planning.

7. Expand Analytical Scope in the Future

While this study focuses on descriptive analysis, future projects can incorporate predictive analytics to forecast delinquency, revenue trends, and customer churn. Advanced analytics can help institutions take preventive actions rather than reactive measures.

6.2 Limitations of the Study

Despite providing meaningful insights, the study has certain limitations that should be considered when interpreting the results.

1. Use of Historical Data Only

The analysis is based entirely on historical data provided in the dataset. It does not include real-time transaction data, which may limit the ability to capture recent changes in customer behavior or market conditions.

2. Absence of Predictive Modeling

The study focuses on descriptive and diagnostic analysis and does not include predictive or machine learning models. As a result, the analysis explains past and current trends but does not predict future outcomes such as default probability or customer churn.

3. Limited External Factors

The dataset does not include external factors such as economic conditions, inflation rates, employment changes, or customer credit history from other financial institutions. These factors can significantly influence spending behavior and delinquency but are beyond the scope of this study.

4. Data Quality Constraints

Although extensive data cleaning was performed, the analysis is still limited by the quality of the original data. Any inaccuracies, missing values, or inconsistencies in the source data may affect the accuracy of the findings.

5. Limited Behavioral Insights

Customer behavior is analyzed based on available transactional and demographic data only. Psychological, lifestyle, and personal preferences that influence spending habits are not captured in the dataset, limiting deeper behavioral analysis.

6. Tool-Specific Constraints

While SQL, Python, and Power BI are powerful tools, each has its own limitations. SQL is less flexible for complex analytics, Python requires technical expertise, and Power BI dashboards depend heavily on the quality of the underlying data model.

Summary

In summary, the recommendations provided in this chapter offer practical guidance for improving credit card operations through better customer segmentation, risk management, and data-driven decision-making. At the same time, the limitations clarify the boundaries of the current study and highlight opportunities for future enhancement. Despite these limitations, the study successfully demonstrates how financial data analytics can support strategic decisions and improve overall business performance.

BIBLIOGRAPHY

Bibliography / References

Books:

1. DATA ANALYTICS AND POWER BI VALUE ADDED COURSE MATERIAL II B.Sc COMPUTER SCIENCE, COMPILED BY M.RIYAZ MOHAMMED
2. Exploratory Data Analysis in Python Version 2.2.0, Allen B. Downey Green Tea Press
3. Head First SQL: Your Brain on SQL -- A Learner's Guide by Lynn Beighley
4. Python for Data Analysis, **Author:** Wes McKinney
5. QL in a Nutshell **Authors:** Kevin Kline, Brand Hunt, & Daniel Kline

Research Paper:

1. McKinney, W. (2010).

   Data structures for statistical computing in Python.

   In *Proceedings of the 9th Python in Science Conference* (pp. 56–61).

2. Chen, H., Chiang, R. H. L., & Storey, V. C. (2012).

   Business intelligence and analytics: From big data to big impact.

   MIS Quarterly, 36(4), 1165–1188.

3. Zamil, M. H., Mohiuddin, M., & Mamun, N. H. (2024).

   Business intelligence systems in finance and accounting: A review of real-time dashboarding using Power BI and Tableau.

   *American Journal of Scholarly Research and Innovation***,** 3(1), 45–58.

4. Russo, M., & Ferrari, A. (2016).

   Introducing DAX as a query language.

   Microsoft Technical White Paper, Microsoft Press.

WEBSITES:

- Kaggle → https://www.kaggle.com
- Dataset → https://www.kaggle.com/datasets/nibeditasahu/credit-card-financial-dashboard-using-power-bi
- MySQL Workbench → https://www.mysql.com/products/workbench/
- Jupyter Notebook (Project Jupyter) → https://jupyter.org
- Python pip documentation - https://pip.pypa.io/en/stable/
- Power Query SQL Server Connector - Power Query SQL Server connector – Microsoft Learn
- Power BI (Microsoft) → https://www.microsoft.com/en-us/power-platform/products/power-bi/
- MySQL Connector/Python - MySQL Connector/Python – Official Download